Computers & Security 159 (2025) 104657

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

DroidReach++: Exploring the reachability of native code in android applications

Luca Borzacchiello [a], Matteo Cornacchia [a], Davide Maiorca [b], Giorgio Giacinto [b], Emilio Coppa [c,*]

[a] Sapienza University of Rome, Italy
[b] University of Cagliari, Italy
[c] LUISS University, Italy

ARTICLE INFO

Keywords:
Android
Static analysis
Mobile security

ABSTRACT

Modern Android applications often incorporate numerous native C/C++ libraries to efficiently handle CPU-intensive tasks or interact at a low level with specific hardware, such as performing specialized GPU rendering. Recent research on Android security has revealed that these libraries are frequently adopted by third-party developers and may pose security risks if not regularly updated, as publicly disclosed vulnerabilities in outdated libraries can be exploited by malicious actors. To determine whether these known vulnerabilities represent an immediate and tangible threat, it is essential to assess whether the vulnerable functions can be executed during application runtime – a research problem commonly known as *function reachability*. In this article, we introduce DroidReach++, a novel static analysis approach for evaluating the reachability of native function calls in Android applications. Our framework overcomes the limitations of existing state-of-the-art methods by combining heuristics with symbolic execution, enabling a more precise reconstruction of Inter-procedural Control-Flow Graphs (ICFGs). When applied to the top 500 applications from the Google Play Store, DroidReach++ identifies a significantly higher number of execution paths compared to previous techniques. Finally, two case studies demonstrate how DroidReach++ serves as an effective tool for vulnerability assessment.

1. Introduction

The Android ecosystem has undergone significant evolution over the years. Applications have become more user-friendly, providing advanced functionalities such as graphics, database management, and modern encryption. Although Java and Kotlin are the languages that are the most used for developing Android apps, higher performance is usually achieved by resorting to C/C++ libraries via the NDK (Native Development Kit). Today, such libraries are widely used by the majority of applications.

As malicious samples typically resort to Java components to carry out their operations (Wong and Lie, 2016; Bello and Pistoia, 2018), there has been a significant research effort aimed at improving the security and integrity of Android apps. The analysis of Java code is crucial to infer the presence of information leaks by employing techniques such as, e.g., taint analysis. In contrast, the focus of research on the Android Native Environment has been limited, mainly due to the evolution of process management security in Android, starting from KitKat (4.4). Before this version, the Android NDK used to be exploited to carry out privilege escalation. With the introduction of ART runtime and security measures such as process isolation, the creation of exploits has become less feasible for attackers. In addition, the interactions between Java and the native layer can be challenging to analyze.

Although the lack of exploitation space may lead readers to believe that potential issues on the native layer do not pose significant security concerns, the Android Native Environment conceals more challenges than superficial assumptions may suggest. In particular, recent work highlighted the significant presence of security flaws in native code (Almanee et al., 2021), mostly due to not up-to-date versions of libraries that are continuously used even in very popular applications (featuring millions of downloads). However, the presence of vulnerabilities alone does not immediately translate into a security risk for applications because it depends on *whether they could be practically exploited*. To address this problem, we investigate the concept of *function reachability*, that is, determining whether functions can be accessed during the application's execution.

* Corresponding author.
E-mail addresses: borzacchiello@diag.uniroma1.it (L. Borzacchiello), cornacchia@diag.uniroma1.it (M. Cornacchia), davide.maiorca@unica.it (D. Maiorca), giacinto@unica.it (G. Giacinto), ecoppa@luiss.it (E. Coppa).

https://doi.org/10.1016/j.cose.2025.104657
Received 8 March 2025; Received in revised form 10 July 2025; Accepted 4 September 2025
Available online 12 September 2025
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Previous research on native code analysis in Android has focused primarily on static (Wei et al., 2018a,b) and dynamic (Xue et al., 2019) analyses in applications that use native libraries. However, such works did not adequately address function reachability. As a consequence, various technical aspects regarding how native functions are reached and their relationship with the Java environment were overlooked. Understanding whether a function can be invoked in Android involves various technical challenges, which cannot be completely solved even with advanced techniques, such as symbolic execution (Shoshitaishvili et al., 2016), which may not scale when considering popular Android apps. Although dynamic analysis features a high level of precision in assessing the reachability of functions, it becomes easily unfeasible for complex applications featuring thousands of classes, leading to the so-called *path explosion* problem. In this sense, it becomes crucial to find a proper balance between the *time* needed for the analysis and the *precision* of the attained results.

In this article, we propose DroidReach++, a static approach to establish the reachability of native methods in Android apps starting from the Java entry points in the application. In particular, we propose the following contributions:

1. We discuss the technical limitations that hinder the analysis capabilities of current analysis tools. In particular, we show limitations in: (a) properly mapping Java `native` methods to Java Native Interface (JNI) methods, (b) handling *nested* native libraries, and (c) accurately building the ICFGs.
2. We introduce the methodology behind our framework, which integrates several heuristics and ICFG construction techniques to mitigate the limitations mentioned in the previous point. DroidReach++ can accurately and effectively reconstruct potential paths to vulnerable native calls.
3. We investigate an interesting execution pattern, dubbed *context-based native execution*, emerging in several Android applications. This pattern involves an application creating a C++ native object through one JNI method, storing its pointer on the Java side, and subsequently passing this pointer to other JNI methods. This allows those functions to access the attributes and virtual methods of the C++ object. However, without tracking these C++ object pointers, accurately reconstructing ICFGs becomes challenging, potentially leading to the omission of critical (and possibly vulnerable) execution paths. To address this, DroidReach++ introduces a novel analysis technique that identifies and leverages these C++ object pointers, enhancing ICFG precision when necessary.
4. We perform an evaluation considering 500 popular applications featuring complex native libraries and ICFGs (with an average of 2,000,000 native instructions and 1,000,000 Java instructions). We show that DroidReach++ can reach more instructions than Argus-SAF (Wei et al., 2018a), which is the state-of-the-art static framework for analyzing the Java and native layers in Android. Furthermore, we offer experimental information on the prevalence of the *context-based native execution* patterns among the applications in the analyzed dataset.
5. We propose two real and practical case studies in which we show how DroidReach++ can be helpful in assessing the reachability of vulnerable functions.

This article constitutes an extension of our previously published work (Borzacchiello et al., 2022). Specifically, we introduce the context-based native execution pattern and explore its application in real applications; we expand and improve the functionality of the previous DroidReach framework to implement this pattern and improve its general functionality on the other patterns; we propose an in-depth analysis on the scalability of the system and its running time performance in comparison to state-of-the-art works. Finally, we further explore the limitations of our approach, sketching intriguing future research directions. To encourage further research, we make our contributions available at https://github.com/season-lab/DroidReach. We believe that DroidReach++ represents a step forward for the Android community as it can provide valuable information to security experts in the presence of large and complex apps.

2. Background and related work

Android Apps. Android applications are typically written in Java or Kotlin, and are compiled into an intermediate bytecode format called Dalvik, with instructions stored in the `classes.dex` files. These files are parsed and converted into native code that is executed by the Android RunTime (ART), allowing for faster execution compared to the previous runtime (Dalvik Virtual Machine), which used a Just-In-Time (JIT) approach without converting the bytecode to native code.

In general, Android applications are zipped `.apk` (Android application package) archives containing the following elements: *(i)* The `AndroidManifest.xml` file and other `.xml` files that provide the application metadata and layout; *(ii)* One or more `classes.dex` files, which contain the executable bytecode of the Java/Kotlin classes; *(iii)* External resources, such as images or native libraries.

The Android NDK. The Android Native Development Kit (NDK) is an ensemble of tools that allows for the implementation of parts of Android apps in native code (C/C++). Such a code is typically employed to guarantee faster performance in comparison to traditional Java code. The interface between the Java and the native layer is called Java Native Interface (JNI). JNI essentially defines how functions receive parameters or provide return values. Native libraries can be loaded with the `System.loadLibrary` method. Then, the native methods to be invoked are declared in the Java code by using the `native` keyword.

Native Libraries Analysis. Native libraries have been extensively studied, particularly in the context of vulnerability identification, i.e., understanding the *presence* of vulnerabilities in native code. Several works have focused on the study of JNI interface vulnerabilities (Tan et al., 2006; Tan and Croft, 2008; Li and Tan, 2009; Gu et al., 2017; Lee et al., 2020; Hwang et al., 2021), as well as the identification of possible vulnerabilities in Android libraries. Derr et al. (2017) conducted a test with 200 developers, showing that many libraries embedded in apps are outdated and contain security vulnerabilities. Various approaches have been proposed to detect these vulnerabilities, including machine learning (Gao et al., 2018), analysis of similarities between functions (Eschweiler et al., 2016; Xu et al., 2020), and hybrid analysis techniques (Ming et al., 2017; Liao et al., 2018).

Almanee et al. (2021) proposed an extended assessment of the presence of vulnerable functions in Android native libraries. In particular, they showed that applications contain libraries that have not been updated even for two years, thus exposing vulnerabilities that typically require a long time to be fixed. We base the beginning of our analysis on the results of this work, as it depicts a critical scenario where various applications may feature critical security issues. Finally, more recent works focused on the detection of vulnerabilities in JavaScript engines embedded in Android applications (Wen et al., 2024).

Dataflow Analysis. Dataflow analysis has been extensively studied in Android, focusing on how data propagates in Java code. This problem has been addressed with static and dynamic approaches. Regarding static approaches, FlowDroid (Arzt et al., 2014) was among the first to introduce the proper handling of Android callbacks. Other works improved FlowDroid in many aspects, such as proper dataflow tracking for intents (Klieber et al., 2014; Gordon et al., 2015; Li et al., 2015; Bosu et al., 2017). Amandroid (Wei et al., 2018b) is considered as the current reference point for static dataflow analysis in Android. Wei et al. (2018a,b) expanded Amandroid by releasing JN-SAF (now known as Argus-SAF), which introduced the analysis of the information flows between the Java and the native layer. The approach employs

symbolic execution to handle the native layer. In particular, Argus-SAF uses `CFGEmulated` from Angr (Shoshitaishvili et al., 2016) to reconstruct ICFGs of the native code (Section 3.1) and compute approximate dataflow facts. Argus-SAF will be the main reference point for the analyses discussed in this paper.

Dynamic approaches employ code instrumentation and execution to perform taint analysis. Droidscope and TaintDroid (Yan and Yin, 2012; Enck et al., 2014) are among the first approaches to adopt this technique. Subsequent works built on and improved TaintDroid (Sun et al., 2016; Xue et al., 2017, 2019) by using, for example, concolic execution (Chen et al., 2013; Borzacchiello et al., 2021b,a). Xue et al. (2017) proposed a dynamic stimulation approach that involves concolic execution (Chen et al., 2013) to explore specific code regions. The concolic execution has been combined with direct execution of methods carried out by using Valgrind APIs to instrument Java methods. NDroid (Xue et al., 2019) is among the first approaches that studied dataflow between the Java and native layers. In particular, the authors identified six scenarios in Java and native environments that would lead to information leakage. Unfortunately, NDroid only supports old Android releases (4.4 and 5.0). A further work that employs dynamic analysis was proposed by Zhang et al. (2023), where machine learning is combined with dynamic analysis by extracting and analyzing function behaviors in context. Unfortunately, generating the *right* executions that will reach a function is especially challenging.

Input Generation. Several works (Android Developers, 2021; Amalfitano et al., 2012; Choi et al., 2013; Hao et al., 2014; Abraham et al., 2015; Wong and Lie, 2016; Yuanchun Li et al., 2017; Wang et al., 2017) aimed at generating *user inputs* that can lead to the execution of specific functions. Systems based on static analysis allow for faster code coverage, but can lack precision. In contrast, approaches based on dynamic analysis can be much more precise, but can often be unfeasible due to the so-called *path explosion* problem. One notable example is Intellidroid (Wong and Lie, 2016), an approach that uses static and dynamic analysis to generate those inputs that allow for reaching specific calls. Intellidroid focuses only on the Java layer without exploring the native layer.

Automatically finding the right set of stimulations for an app remains an open research problem, especially when considering large and complex apps. DroidReach++ cannot find the inputs able to reproduce a specific path but can provide insights about the existence of a path towards a specific target point.

Overall, as presented in this section, several state-of-the-art works have implicitly or explicitly tackled reachability analysis in the context of Android applications since this is a pivotal task for a large number of security tasks. However, mature frameworks such as FlowDroid and Amandroid do not take into account the native libraries of Android applications, potentially missing several interesting execution paths. While there are several works that focus on native library analysis for traditional applications, they do not consider the specific nuances deriving from the Android platform and its development process. Nonetheless, Argus-SAF has been the first to focus on the tight interaction between Java code and native code. For this reason, in the remainder of this article, we consider Argus-SAF as our main competitor. Unfortunately, as we will motivate in the following section, Argus-SAF comes with some limitations when considering the reachability challenges emerging in modern Android applications.

3. DroidReach

In this section, we present the main ideas behind our framework - DroidReach++. First, we discuss the problem targeted by DroidReach++ and the challenges that affect existing approaches. Then, we present the design and components of DroidReach++.

3.1. Terminology and problem statement

Terminology. In the following, we define a few terms used throughout the article:

- A **code point** p for our analysis is an instruction within the set of instructions from the Java layer (J) or the native layer (N) of an app, i.e., $p \in (J \cup N)$. For the sake of simplicity, in our discussion we often use the term *method* to refer to the first instruction of the method.
- A **Control-Flow Graph (CFG)** is a graph representation of possible paths that can be taken during the execution of a function. Each node represents a contiguous sequence of code points. Edges represent jumps across nodes.
- A **Call Graph (CG)** is a graph that represents the calls across different functions of an app.
- An **Inter-procedural Control-Flow Graph (ICFG)** connects the CFGs of different functions using the information from the CG. In practice, it can encode all the application's paths starting from a specific entry point.
- A **source** p_s is a code point in an app that could start the execution of some Java code. Hence, a source could be seen as an entry point for the Java layer.
- A **sink** p_t is a code point inside a native library, i.e., $p_t \in N$. The sink identifies an interesting point that we would like to reach during the execution.

Goal. Given the Java instructions J, the native instructions N, and a sink p_t, our goal is to identify at least one path starting from a source p_s and ending in the sink p_t. The path is represented as the sequence of points traversed in the ICFG from the source p_s. Identifying a path in the ICFG can be valuable for several program analyses and security tasks. This paper focuses on sinks that could be associated with vulnerable code points within native libraries.

3.2. Reachability challenges

Argus-SAF (Section 2) is the state-of-the-art solution to statically analyze the Java and the native layers of an Android application. When testing it on real-world applications for our goals, we have identified a few critical challenges that affect its accuracy (and also that of other existing work):

C1 **Mapping Java native methods to JNI methods.** To execute the code of a native library, the Java code invokes a Java `native` method. Java `native` methods at running time could be seen as *jumps* to JNI methods, which are the *entry points* for the native layer. Unfortunately, statically identifying the *mapping* between Java `native` methods and JNI methods is not always trivial. State-of-the-art solutions may fail (Section 4) to *resolve* a large number of these mappings, possibly ignoring several entry points of the native layer.
Examples. Real-world examples of static and dynamic mappings are provided in Figs. 1 and 2, respectively. In particular, Fig. 1 shows how the java class `EffectFragment` declares a `native` method `shadows`, which is then defined by the native code. Interestingly, the function in the native code follows a complex naming convention: this is needed to *statically* map the native implementation with its related Java `native` method. On the other hand, Fig. 2 provides an example of a *dynamic* mapping: the native function `n_getEffects` does not follow the complex naming convention mentioned above and is explicitly mapped to the java `native` method `n_ApplyActions` through the execution of the `JNI_OnLoad` function, which is a special init handler specifically proposed by JNI to cope with dynamic mappings.

C2 ***Nested* native libraries.** A JNI method is part of a shared library, e.g., `libA.so`, which is nothing more than a traditional shared library. As in most platforms, a shared library is free to call methods of other libraries, i.e., one library may link another one (e.g., `libB.so`). State-of-the-art solutions may not perform analyses across multiple native libraries in the case of *nesting*. This is crucial on Android, since applications may often: (a) integrate open-source libraries, which may rely on other ones, and (b) devise *wrappers* to work with libraries that were not originally written for Android and thus do not implement the JNI API.
Example. Fig. 3 shows a real-world dependency chain among native libraries: the java package `com.tel komsel.telkom selcm` makes use of the native library `libjniPdfium.so`, which in turn relies on functions from the native library `libmodpdfium.so` that exploits functions from the native library `libmodft2.so`.

C3 **Scalability versus accuracy.** Argus-SAF uses symbolic execution to analyze the native code. While this technique can be very accurate during the ICFG construction, allowing the tool to even compute data flow facts, it does not scale on complex libraries. Indeed, Argus-SAF trades accuracy for scalability, halting its analysis when the call depth is larger than, e.g., 5, which is not enough in several cases, thus generating incomplete ICFGs. Approaches based on traditional binary frameworks (NSA, 2016), may provide better scalability but then generate less accurate ICFGs, e.g., in the presence of indirect jumps.

C4 **Context-based native execution.** JNI methods may take as arguments data that other JNI methods may have generated. This aspect is common, e.g., in the case of C++ code: non-static class methods take the `this` pointer as an argument, using it when calling *virtual* class methods. Hence, several JNI methods should be analyzed based on an *execution context*. However, Android applications do not ship with the source code for the native libraries and often even remove debugging information from the binary code, making it extremely hard to understand the execution context of a JNI method. In practice, existing solutions may fail to analyze native calls whose target depends on the execution context.
Example. Fig. 4 provides a real-world example of context-based native execution. In particular, the Java method `Builder` constructs a C++ object using the Java method `createObj`, which internally uses a statically mapped native function from `libcomScore.so` to instantiate a C++ object. In this article, we refer to such a native function as an *object producer*. Interestingly, since Java has no built-in type for C++ object pointers, the Java code is forced to cast the pointer obtained from the native side to a `long` type. Such a casting makes it hard to identify pointers when monitoring the Java execution. After creating the C++ object and storing it into the `cppObj` attribute, the Java code exploits the pointer within the Java method `relatedContentMetadata`, which invokes the `native` method `relatedContentMetadataNative` that is statically mapped to the second function provided by `libcomScore.so`. Such a native function *consumes* the pointer to call several C++ virtual methods and thus, in this article, is dubbed *object consumer*.

To further help the reader grasp the technical aspects behind these challenges, we will explore them in the context of a running example in the remainder of the section. However, we first present at a high level the design behind DroidReach++.

3.3. Architecture of DroidReach++

Fig. 5 depicts the main steps taken by our framework, in relation to the challenges proposed in Section 3.2:

```
// Java layer: classes.dex
package com.lyrebirdstudio.lyrebirdlibrary;
public class EffectFragment extends Fragment {
  private static native void shadows(Bitmap arg0, float arg1)
  { /* statically resolved by dlsym() based on the name of the JNI method */ }

// Native layer: libfilter.so
void Java_com_lyrebirdstudio_lyrebirdlibrary_EffectFragment_shadows(
  JNIEnv *env, jclass c, jfloat arg1) { /* native implementation of the JNI method */ }
```

Fig. 1. [C1] A statically defined JNI method.

```
// Java layer: classes.dex
package com.aviary.android.feather.headless.moa;
public class Moa {
static native void n_applyActions() { /* dynamically resolved by JNI_OnLoad() */ }

// Native layer: libaviary_native.so
JNIEXPORT jint JNI_OnLoad(JavaVM* vm, void* reserved) { ...;
 jclass c = env->FindClass("com/aviary/android/feather/headless/moa/Moa");
 static const JNINativeMethod m[] =
{ ..., {"n_getEffects", "()[Ljava/lang/string;]", (void*)(n_getEffects)}, ... };
 env->RegisterNatives(c, m, sizeof(m) / sizeof(JNINativeMethod)); ...; }

jobjectArray n_getEffects(JNIEnv *env, jclass c)
  { /* native implementation of the method */ }
```

Fig. 2. [C1] A dynamically defined JNI method.

S1 **Static analysis of the Java layer.** The first step builds the ICFGs of the Java code, identifying sources and calls to Java `native` methods.

S2 **Analysis of interactions between Java and native layer.** After detecting the Java `native` methods that could be reached during an execution, DroidReach++ identifies the mappings between Java `native` methods and JNI methods, which are the entry points for the native layer. This step is therefore designed to tackle challenge C1.

S3 **Static analysis of the native layer.** Given a list of JNI methods that could be reached during the execution, DroidReach++ builds the ICFGs of the native libraries. This step aims at challenges C2 and C3, combining different techniques to achieve a nice trade-off between accuracy and scalability.

S4 **Analysis of native context-based executions.** To further improve the accuracy of native ICFGs, DroidReach++ develops a technique to heuristically cope with challenge C4. In particular, it identifies JNI methods returning C++ objects to the Java layer and propagates information about the C++ virtual tables to JNI methods with a context-based execution.

S5 **Reachability analysis.** The last step is where our framework assembles the pieces constructed in the previous stages. It first merges the ICFGs of the Java layer with the ICFGs of the native layer and then evaluates whether there exists a path from a source p_s to a user-defined sink p_t.

In the remainder of this section, we review in detail these steps. Throughout our discussion, we use a running example: Fig. 6 shows an excerpt of its code. This is an app with two activities, where the first one (`LoginActivity`, omitted from the code) checks the credentials of the user, while the second one (`JavaLayerActivity`) runs some tasks using some native libraries (`native-lib .so` and `other-native-lib.so`) when the user clicks a button. Unlike dynamic approaches, DroidReach++ can focus directly on `JavaLayerActivity` without necessarily satisfying the execution requirements of `LoginActivity`, which could be arbitrarily difficult to automatically identify and satisfy.

3.4. Static analysis of the Java layer

Different state-of-the-art frameworks already exist to analyze the Java layer, providing different trade-offs in terms of accuracy and

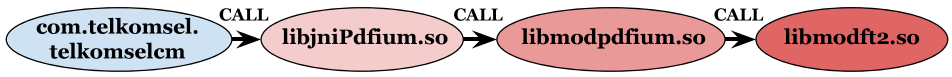

Fig. 3. [C2] Example of *nested* libraries.

```
// Java layer: classes.dex
package com.comscore.streaming;
public class AdvertisementMetadata extends AssetMetadata {
 public static class Builder extends CppJavaBinder { ...
  public Builder() { ...; this.cppObj = AdvertisementMetadata.createObj(); ... }

  static long createObj() {
   // it calls a java native method that returns a C++ object
   return AdvertisementMetadata.newCppInstanceBuilderNative(); }

  public Builder relatedContentMetadata(ContentMetadata arg) { ...;
   // the cppObj field is passed as an argument
   AdvertisementMetadata.useCppObj(this.cppObj, arg.b());
   ...; return this; }

  static void useCppObj(long cppObj, long arg2) {
   // the C++ object ptr is passed to a java native method
   AdvertisementMetadata.relatedContentMetadataNative(cppObj, arg2); }

// Native layer: libcomScore.so
jlong Java_com_comscore_streaming_AdvertisementMetadata_newCppInstanceBuilderNative(
            JNIEnv *env, jclass c)
 { /* it returns a C++ object. It is an "object producer" */ }

void Java_com_comscore_streaming_AdvertisementMetadata_relatedContentMetadataNative(
    JNIEnv *env, jclass c, jlong cppObj, jlong arg2)
  { /* it calls a virtual C++ method of cppObj. It is an "object consumer" */ }
```

Fig. 4. [C4] Example of a context-based native execution.

scalability. The current implementation of DroidReach++ can work with Androguard (Desnos, 2021) and FlowDroid (Arzt et al., 2014). Regardless of the specific framework in use, DroidReach++ performs three stages:

1. **Identification of sources.** DroidReach++ looks for sources by considering the class methods of several Android components, following the guidelines and suggestions proposed in previous works (Desnos, 2021; Arzt et al., 2014). In particular, our framework considers the following components:

   - *Activities*: they can be defined as windows (full- screen or floating) that interact with the user when the app runs.
   - *Providers*: components that provide access to a central repository of data, in a similar way to database tables.
   - *Services*: components that can perform background operations during the execution of the app.
   - *Receivers*: components that activate when receiving specific messages from the operating system.

   Such components are essential for the user's interaction with the application. Thus, they can be considered as possible sources from which native methods can potentially be reached.
2. **ICFG construction.** For each source, DroidReach++ builds an ICFG considering the entire Java code of the app, connecting the CFGs of the methods based on their *caller-callee* relationships.
3. **Identification of Java native methods.** Finally, this step identifies Java `native` methods which are invoked in the ICFGs of the Java layer. At this stage, a call to a Java `native` method is not yet mapped to its JNI method, which contains the actual binary implementation of the Java `native` method.

Running example. When considering the `JavaLayer- Activity` class, there are two sources: `onCreate` (J3), which is executed at the activity start, and the `onClick` handler (J8) for button events. Fig. 7 shows in blue the Java code points for the ICFGs starting from these two sources. Notice how nodes J13, J14, J15, and J16 are Java `native` methods. An additional *implicit* source, considered by DroidReach++ but omitted from Fig. 7, is J1, which triggers the execution of `JNI_OnLoad` (see next section).

3.5. Analysis of interactions between Java and native layer

Each Java `native` method is mapped to a JNI method in the native layer. The mapping can be defined statically or dynamically, as described in the following.

Static mapping of JNI methods. The name of the JNI method is a symbol exported by the library that follows a specific mangling scheme, allowing the dynamic linker to uniquely identify the Java `native` method associated with it. For instance, the native method `com.lyrebirdstudio.lyrebirdlibrary.EffectFragment.shadows` in Fig. 1 maps to the JNI method `Java_com_lyrebirdstudio_lyrebirdibrary_EffectFragment_shadows`. In more detail, when a method is declared as `native` in Java, the JVM expects the corresponding native function to follow a specific naming pattern: `Java_`*<fully_qualified_class_name>*`_`*<method_name>*. For example, if a class `MyClass` contains a native method `myMethod`, the corresponding C function must be named `Java_MyClass_myMethod`. If the class is within a package, each dot in the package name is replaced by an underscore; for instance, the method `myMethod` in class `com.example.MyClass` becomes `Java_com_ example_MyClass_myMethod`. Additionally, if the native method is overloaded in Java, the JVM distinguishes between them by appending a double underscore followed by an encoded representation of the method's parameter types (known as the JNI signature). This ensures that each native method has a unique name, even if multiple overloads exist in the Java source.

As in previous state-of-the-art works (Wei et al., 2018a), Droid Reach++ uses a decoder of the mangling scheme to resolve statically defined JNI methods.

Dynamic mapping of JNI methods. When the dynamic loader loads a library, it runs the `JNI_OnLoad` function exported by the library. This function may dynamically define mappings between Java methods and native functions using the JNI primitive `RegisterNatives`, which takes as one of its arguments a pointer to an array of `JNINativeMethod`. This `struct` is defined as the following:

```
typedef struct {
  char *name;    // ex: "nativeCtor"
  char *sign;    // ex: "(Ljava/lang/String;)J"
  void *fnPtr;    // function code pointer
} JNINativeMethod;
```

The `struct` states that the native implementation of the Java native method `name` having the signature `sign` (which defines, in `smali`, the types of the method arguments and the return value) is available at the address `fnPtr`. Fig. 2 shows how a real-world application is defining the mapping for the JNI method `n_getEffects`.

Previous works (Wei et al., 2018a) perform symbolic execution from `JNI_OnLoad` to identify the array passed to `RegisterNatives`. This strategy has two downsides: (a) the exploration is halted when reaching a given call depth (e.g., 5 in Argus-SAF) to mitigate path explosion, possibly failing to reach `RegisterNatives`, and (b) the exploration may incur a large overhead when `JNI_OnLoad` is not trivial.

For these reasons, our approach devises a more scalable heuristic to detect dynamic mappings. The key idea is that several developers follow the guidelines of Android (Android, 2021) and statically define

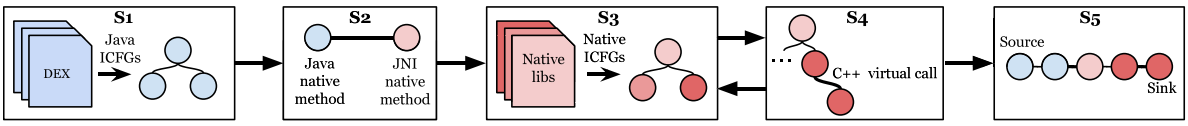

Fig. 5. Main steps of DroidReach++.

```
// Java layer: classes.dex
    public class JavaLayerActivity extends
        AppCompatActivity implements View.OnClickListener {

J1.  static { System.loadLibrary("native-lib"); }

J2.  private long native_obj1;

J3.  protected void onCreate(Bundle state) {
J4.    super.onCreate(state);

J5.    native_obj1 = build_obj1();

J6.    setContentView(R.layout.activity_native_buttons);
J7.    findViewById(R.id.btn_1).setOnClickListener(this);
     }

J8.  protected void onClick(View view) {

J9.   int v = own_class_method(obj1);
J10.  v += parent_class_method(obj1);

J11.  findViewById(R.id.text_1).setText(String valueOf(v));

J12.  execMethod();
     }

     // static mapping
J13. public native long build_obj1();
J14. public native int own_class_method(long o1);
J15. public native int parent_class_method(long o1);

     // dynamic mapping
J16. private static native void execMethod();
    }

// Native layer: native-lib.so
N0. JNIEXPORT jint JNI_OnLoad(JavaVM* vm, void* reserved) {
     JNIEnv* env; vm->GetEnv(&env, JNI_VERSION_1_6);
     jclass c = env->FindClass("JavaLayerActivity");
     static const JNINativeMethod mappings[] =
        { {"execMethod", "()", execMethod} };

     /* some complex code */

     return env->RegisterNatives(
        env->FindClass("JavaLayerActivity"),
        mappings, sizeof(methods)/sizeof(JNINativeMethod));
    }
```

```
// Native layer: native-lib.so (cont'd)
     class C1 {
N1.   public: virtual int M1(){ return 1; }
     };

     class C2 : public C1 {
N2.   public: virtual int M2(){ return 2; }
     };

N3.  JNIEXPORT jint JNICALL
      Java_com_example_JavaLayerActivity_build_obj(
            JNIEnv *env, jobject thiz, jlong obj) {
N4.    return (long)(new C2()); }

N5.  JNIEXPORT jint JNICALL
      Java_com_example_JavaLayerActivity_own_class_method(
             JNIEnv *env, jobject thiz, jlong obj) {
N6.    return ((C2 *)(obj))->M2();
     }

N7.  JNIEXPORT jint JNICALL
    Java_com_example_JavaLayerActivity_parent_class_method(
            JNIEnv *env, jobject thiz, jlong obj) {
N8.   return ((C2 *)(obj))->M1(); }

N9.   static void execMethod(JNIEnv* env, jclass clazz) {
N10.   Handler* h = build();
N11.   h->callback(); // use of fn pointer
      }

N12.  Handler* build() {
N13.   auto* h = new Handler();
N14.   h->init(); // virtual call
N15.   return h; }

N16. void Handler::init() { this->callback = &foo1; }

// Native layer: other-native-lib.so
N17. void foo1() { foo2(); }
N18. void foo2() { foo3(); }
N19. void foo3() { foo4(); }
N20. void foo4() { bug(); }
```

Fig. 6. Running example.

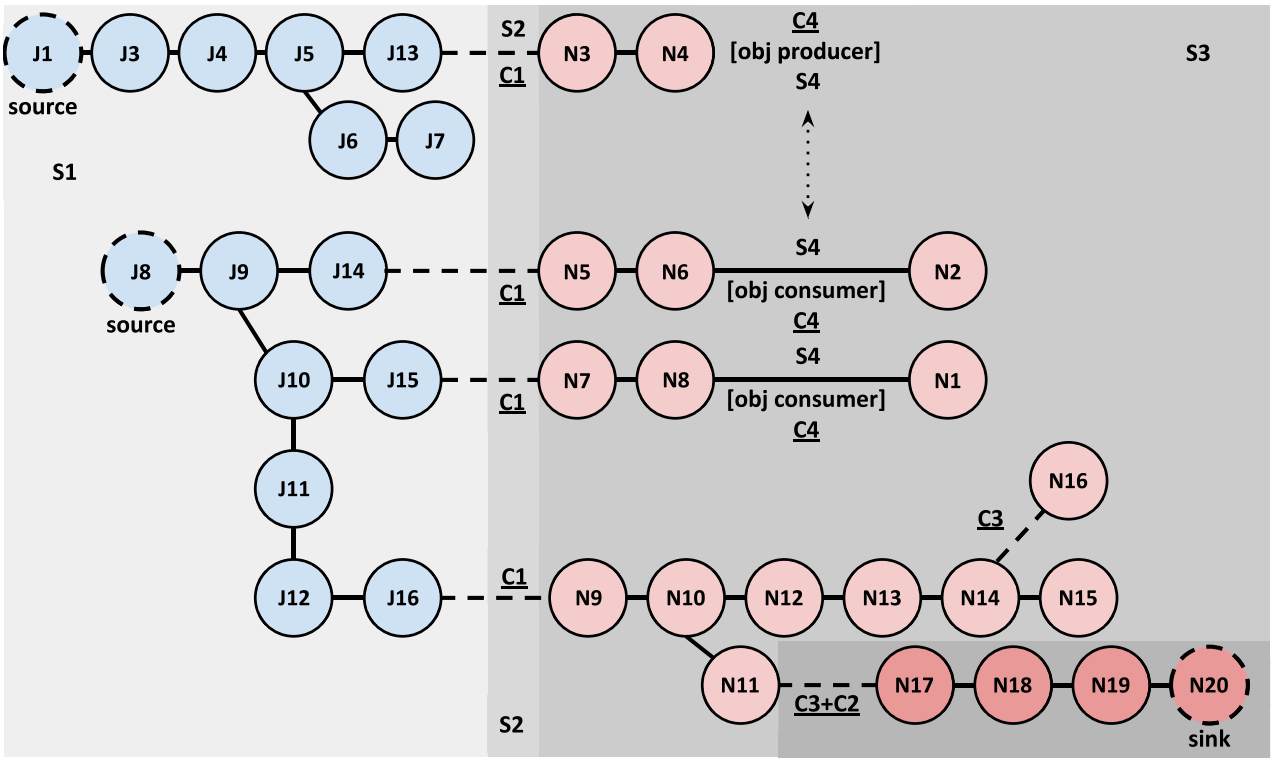

Fig. 7. ICFGs of the running example: shades of gray highlight different steps. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the `JNINativeMethod` array at compilation time, placing it in the global data section.

As illustrated in Algorithm 1, DroidReach++ linearly scans the code sections of a library binary, looking for an array containing elements following the pattern:

1. Pointer to a valid string (`name`): DroidReach++ checks whether the pointer is within a valid section and that a valid string can be extracted at the obtained offset.
2. Pointer to a valid string (`sign`): DroidReach++ checks whether the pointer is within a valid section and that a string can be extracted at the obtained offset whose content is compliant to a valid arguments' signature.
3. Pointer to a function in the `text` section (`fnPtr`): DroidReach++ checks whether the pointer is within a valid code section.

Since some apps may instead allocate and initialize the array during the execution of `JNI_OnLoad`, DroidReach++ fallbacks to symbolic execution when:

1. The above heuristic fails to identify any mapping for a library containing `JNI_OnLoad`.
2. The heuristic identifies some mappings, but there are clashes on the pair (`name`, `signature`) of some methods, e.g., there are multiple Java methods from different classes called `name` with the same `signature`, requiring to inspect additional arguments of `RegisterNatives` to solve the ambiguity.

Hence, DroidReach++ reverts to a more heavyweight analysis based on symbolic execution only when there is an indication that the heuristic is not working correctly.

Running example. When loading the `JavaLayerActivity` class, the loader is executed due to `J1`, processing `native-lib.so`.

First, DroidReach++ identifies the static mappings (`J13`, `N3`), (`J14`, `N5`), and (`J15`, `N7`).

Next, the `JNI_OnLoad` function of `native-lib.so` is considered to recover dynamic mappings, allowing DroidReach++ to recover the mapping (`J16`, `N9`) thanks to the identification in memory of the array `mappings`.

Notice that symbolic execution analyses over `JNI_OnLoad` may struggle to recover the mapping (`J16`, `N9`) in the presence of non-trivial code before the call to `RegisterNatives`.

3.6. Static analysis of the native layer

After identifying the JNI mappings, DroidReach++ constructs the ICFGs for the native layer considering each JNI method as a possible entry point. To cope with challenge C2, DroidReach++ recursively builds the ICFG if a function of a library calls a function of another library. In addition, to cope with challenge C3, our approach combines the ICFGs built by different techniques.

The high-level workflow of this analysis phase is shown in Algorithm 4, which iterates on each JNI method and builds the ICFG of a native library from the considered entry point (i.e., the JNI method). The algorithm first identifies the library that contains the JNI method and then calls the AnalyzeNatLib function to build the ICFG of the library. The AnalyzeNatLib function, shown in Algorithm 3, builds the ICFG of a library by calling the GetICFG function, shown in Algorithm 2, which is responsible for building the ICFG of a given JNI method. The AnalyzeNatLib function is recursive, as it may need to build the ICFG of methods never met before (or it may discover new calls to other imported functions). We now review in detail the stages of this step.

ICFG construction. For each JNI method, as show in Algorithm 2, DroidReach++ builds the CFGs and the CGs of native functions to obtain the ICFGs. Our implementation uses the Ghidra reversing framework (NSA, 2016), as it worked particularly well when considering libraries found in Android applications. The ICFGs derived in this stage

Algorithm 1 FindJniMethod

```
procedure FindJniMethods(lib)
    let P_SZ ← size of a pointer
    let methods ← {}
    for all section in lib.sections do
        for all offset in section do
            namePtr ← ReadPtrAt(offset)
            if not ValidPtr(namePtr, lib.sections) then
                continue
            end if
            name ← ReadStr(namePtr, lib.sections)
            if not validString(name) then
                continue
            end if
            signPtr ← ReadPtr(offset + P_SZ)
            if not ValidPtr(signPtr, lib.sections) then
                continue
            end if
            sign ← ReadStr(signPtr, lib.sections)
            if not validString(sign) then
                continue
            end if
            if not ValidArgsSignature(sign) then
                continue
            end if
            codePtr ← ReadPtr(offset + 2 · P_SZ)
            if not valCPtr(codePtr, lib.sections) then
                continue
            end if
            methods[offset] ← {name, sign, codePtr}
        end for
    end for
    return methods
end procedure
```

Algorithm 2 GetICFG

```
1: procedure GetICFG(lib, addr, icfgs)
2:     if lib ∈ icfgs then
3:         if addr ∈ icfgs[lib].entries then
4:             return icfg
5:         end if
6:     end if
7:     cfg ← BuildCFGwithGhidra(lib, addr, icfgs)
8:     cfg ← RefineWithAngr(cfg, lib, addr, icfgs)
9:     cg ← BuildCGwithGhidra(lib, addr, icfgs)
10:    icfgs ← BuildICFG(cfg, cg, addr, icfgs)
11:    icfgs[lib].entries ← icfgs[lib].entries ∪ { addr }
12:    return icfgs
13: end procedure
```

include only code points from the same shared object of the starting JNI method.

Library dependency graph. Given the ICFG for a JNI method, our approach analyzes the *calls* to imported functions, i.e., calls to functions from other libraries. To represent this information for all ICFGs, it defines a *library dependency graph*, where the nodes represent libraries, and the edges are calls across different libraries. Each edge is annotated with a list of caller-callee tuples to track the different calls that may involve the same pair of libraries.

ICFG refinement: nested libraries. Exploiting the library dependency graph, DroidReach++ refines the ICFG of each JNI method to include code points from nested libraries. In particular, as shown by lines 8–12 in Algorithm 3, it links CFGs of different libraries by adding

Algorithm 3 AnalyzeNatLib

```
1: procedure AnalyzeNatLib(lib, addr, icfgs)
2:     if lib ∈ icfgs then
3:         if addr ∈ icfgs[lib].entries then
4:             return icfgs
5:         end if
6:     end if
7:     icfgs ← GetICFG(lib, addr, icfgs)
8:     libDeps ← FindLibDeps(icfg)
9:     for all (src, dst, libDep) in libDeps do
10:        icfgs ← AnalyzeNatLib(libDep, dst, icfgs)
11:        icfgs ← AddEdges(icfgs, src, dst, lib, libDep)
12:    end for
13:    return icfgs
14: end procedure
```

Algorithm 4 AnalyzeNativeLayer

```
1: procedure AnalyzeNativeLayer(jniMethods, libs)
2:     icfgs ← {}
3:     for all jniMet in jniMethods do
4:         lib ← FindLib(jniMet, libs)
5:         icfgs ← AnalyzeNatLib(lib, jniMet, icfgs)
6:     end for
7:     return icfgs
8: end procedure
```

call and *return* edges whenever a function from a library calls a function of another library. Since this stage may need to build the ICFG of methods never met before, (or it may discover new calls to other imported functions), our approach iteratively repeats the two previous stages until a fixed point is reached.

ICFG refinement: symbolic exploration. The previous stages can build ICFGs that may traverse several libraries, potentially representing paths able to reach even *deep* code points in an execution path. However, the previous stages may still miss some critical edges in the ICFG: e.g., in the presence of a callee that performs an indirect call using a target defined by its caller. Although reverse engineering frameworks, such as Ghidra, have significantly reduced the need for heavyweight dataflow analyses, there are still several cases where they may be needed to accurately build an ICFG. For this reason, for each JNI method, as shown by line 8 in Algorithm 2, our approach performs a symbolic exploration using `CFGEmulated` of angr (Shoshitaishvili et al., 2016) to recover the missed edges. To control path explosion, the exploration stops its analysis when the call stack contains more than 5 nested calls (as in Argus-SAF). Moreover, path branches are not evaluated, thus skipping most symbolic queries. After running this refinement step, we repeat the two previous stages until a fixed point is reached. Hence, DroidReach++ combines two different techniques for building ICFGs: the first is more scalable but less accurate, while the second is more accurate but less scalable. Previous approaches, such as Argus-SAF, have favored ICFG construction approaches based on symbolic execution that, however, can struggle at reaching *deep* code points.

Running example. Starting from `N3`, DroidReach++ builds one ICFG with code points {`N3`, `N4`}. Next, from `N5`, it builds an ICFG with {`N5`, `N6`}. From `N7`, it builds an ICFG with {`N7`, `N8`}. Finally, from `N9`, it builds an ICFG with {`N9`, `N10`, `N12`, `N13`, `N14`, `N15`, `N11`}.

Indirect jumps (dashed edges) starting from `N11` and `N14` are not discovered by Ghidra but can be recovered using a symbolic exploration. Since `N11` calls the function at `N17` within `other-native-lib.so`, DroidReach++ builds the library dependency graph, analyzes this library, and adds {`N21`, `N18`, `N19`, `N23`} to the ICFG.

Algorithm 5 IdentifyConsumerArgs

```
1: procedure IdentifyConsumerArgs(jniMethod)
2:     ptrs ← {}
3:     for each arg in jniMethod.args do
4:         if arg.type is long then
5:             addr ← AllocDummyVTable()
6:             arg.value ← addr
7:             ptrs[addr] ← arg
8:         end if
9:     end for
10:    states ← emulate(jniMethod, targets=ptrs)
11:    consumer_args ← []
12:    for each state in states do
13:        if state.pc ∈ ptrs then
14:            consumer_args.append(ptrs[state.pc])
15:        end if
16:    end for
17:    return consumer_args
18: end procedure
```

Even when exploiting symbolic execution, our framework is unable to recover the virtual table of the object `obj1` passed as an argument to `own_class_method` (`J14` on the Java layer and `N5` on the native layer) and `parent_class_method` (`J15` and `N7`). Hence, the indirect jumps at `N6` and `N8`, are left *unresolved*. This is because the object is passed as a `long` value, without any clue about its methods and virtual table. This issue is what we recognized in challenge C4 (Section 3.2), which will be addressed in the next section.

3.7. Analysis of context-based JNI executions

Previous steps look at each JNI method in isolation. However, one JNI method can take as an argument one object that was built by another JNI method. Since a JNI method may invoke methods on the received object, it could be crucial to recover at least the *virtual table* of the object, i.e., the table listing the object's methods. Unfortunately, in several Android applications, object pointers can be stored in the Java layer using a `long` data type (as suggested by the Android documentation (Documentation, 2021), without providing any clue about the object methods. DroidReach++ thus devises an analysis to identify object *producers*, which construct a C++ object and return it to the Java layer, and object *consumers*, which make calls to object methods.

During the formulation of our analysis, we investigated whether existing solutions (Oxgalz, 2021; sirmabus, 2021; Strelsky, 2021; Pawlowski et al., 2017; Schwartz et al., 2018) for C++ binary reversing may help us to cope with challenge C4. There are five main reasons why these solutions are not effective in the context of Android native libraries: (a) the libraries may be stripped, removing valuable information about the methods' signature, (b) the libraries may not contain run-time type identification (RTTI), preventing easy detection of the virtual tables, (c) an object consumer may invoke JNI primitives which are not handled correctly by these frameworks, (d) they are unable to evaluate execution steps performed in the Java layer, thus possibly missing crucial information (e.g., which is the constructor of an object received by a method), and (e) they exploit low-level details about how the compiler generates C++ objects and thus support only a limited set of platforms (excluding, e.g., Android).

Identification of native object *consumers*. As show in Algorithm 5, DroidReach++ first identifies which JNI methods are (likely) receiving a C++ object from the Java layer and invoking one of its methods during the execution. To this aim, DroidReach++ considers any JNI method which takes as one of its arguments a `long`, i.e., a Java data type that could be used in the Java layer to store an object pointer. Given such JNI method, DroidReach++ computes alternative

Algorithm 6 IdentifyProducer

```
1: procedure IdentifyProducer(jniMethod)
2:     if jniMethod.return_type is not long then
3:         return False
4:     end if
5:     trackAllocs ← True
6:     states, allocs ← emulate(jniMethod, trackAllocs)
7:     for each state in states do
8:         if state.return_value ∈ allocs then
9:             return True
10:        end if
11:    end for
12:    return False
13: end procedure
```

paths traversing *unresolved* indirect calls and then runs a lightweight dataflow analysis, based on symbolic execution, to emulate these paths (line 10 in Algorithm 5). This analysis initializes the `long` argument with a *sentinel* value S, which emulates a fake dummy virtual table (see line 5 in Algorithm 5), and then checks whether a path is performing these three actions:

1. Reading a value X in memory at an address *near* S;
2. Using X as a base pointer of an array to retrieve a value Y;
3. Jumping at Y.

These actions represent, at a binary level, what a function would do to call a virtual method of a C++ object: given the address S of the object, it would recover near S the address X of the virtual table, and then it would jump into the method Y after recovering its address from the virtual table. The emulation is mainly designed to propagate the value S and thus should scale better than, e.g., an exploration based on symbolic execution. From an implementation point of view, the use of the dummy virtual table can be assessed by checking the program counters after the emulation (see lines 12–16 in Algorithm 5).

It is important to note that the function pointer at Y may not be extracted solely from the main object's virtual table. Depending on the layout of the C++ native object, steps 1 and 2 may be repeated in succession to extract the final pointer Y from other parameters, including nested classes with their own virtual tables or other sources of function pointers. In these cases, DroidReach++ is capable of following these dereferences recursively until eventually reaching step 3, although extracting the correct virtual table in the following steps may not always be possible depending on the accuracy of the emulation within the producer and the configurable maximum recursion depth.

Identification of native object *producers*. After identifying object consumers, DroidReach++ attempts to detect object producers, i.e., JNI methods returning C++ objects. To this aim, as shown in Algorithm 6, DroidReach++ considers JNI methods returning a `long` value, and that calls the `new` operator, i.e., they allocate memory during their execution. It then emulates their paths while:

1. Replacing each object pointer with a sentinel value.
2. Propagating the sentinel values to detect whether the JNI method is returning one of them to its caller.

Suppose one sentinel value is returned by a JNI method. In that case, DroidReach++ inspects the memory associated with the sentinel to recover the virtual table and marks the method as a producer.

Mapping producers to consumers. Next, as shown in Algorithm 7, DroidReach++ attempts to pair an object producer with an object consumer. The current implementation exploits a heuristic (SortByCP in Algorithm 7): given a consumer, DroidReach++ first attempts to pair it with a producer from the same Java class and then, in case of failure, with one from the same native library. The pair is believed to be valid

Algorithm 7 MapC2P

```
1: procedure MapC2P(consumer, producers)
2:     P ← SortByCP(consumer.class, producers)
3:     for each prod in P do
4:         if ValidateVTable(prod, consumer) then
5:             return prod
6:         end if
7:     end for
8:     return None
9: end procedure
```

when all virtual calls from the consumers can be resolved using the virtual table found by the producer.

ICFG refinement: native object consumers. Whenever DroidReach++ is able to identify a pair of (producer, consumer) methods, it refines the ICFG of the consumer by repeating the path emulation performed in the first stage, propagating the information about the location of the virtual table towards unresolved indirect calls. To further improve the accuracy, our implementation interleaves steps S3 and S4, exploiting the knowledge of the virtual table during the symbolic exploration.

Running example. When analyzing the indirect jumps N6, and N8, DroidReach++ marks these code points as *potential object consumers* since they are *unresolved* jump targets. Similarly, when analyzing `build_obj()` at J13, our framework DroidReach++ marks this code point as a *potential object producer* since it returns a `long` value. Hence, DroidReach++ executes a symbolic emulation starting at N3, detecting the virtual table associated with the `long` value returned by the function. To exploit such discovery, DroidReach++ then starts a symbolic emulation at N5 and N7, successfully recovering the targets of the indirect jumps at N6 and N8. Finally, it adds the code points N1 and N2 to the ICFG. Notice that our implementation can cope with different nuances of C++ objects, including calls to virtual methods of the current class, such as N2, or from a parent class, such as N1.

3.8. Reachability analysis

The last step is in charge of computing a path from a source to a sink.

Defining the sink. In general, the sink is a user-defined choice that is tightly connected to the goal targeted by an analysis. In this paper, given a vulnerability report, we define the sink as the *closest* code point (or even the set of code points if there is not a unique choice) that the app execution should reach in order to reproduce the bug described in the report. To identify the open-source project related to a library from an Android app, including the adopted release, we refer to solutions, such as Almanee et al. (2021), that have proposed effective binary similarity techniques.

Merging ICFGs. Given a sink, DroidReach++ exploits the JNI mappings to connect ICFGs of the Java layer to the ICFGs of the native layer.

Finding a path from a source to a sink. Finally, for each source p_s and for each sink p_t, our approach evaluates whether there exists a path from p_s to p_t. In practice, since there could be several alternative paths between p_s and p_t, our current implementation by default emits the shortest one as it typically is the simplest to check for a user. However, alternative paths can be requested. DroidReach++ does not offer the ability to directly provide the user inputs or other environmental circumstances required to trigger such functions explicitly, but knowledge of the potential sources greatly simplifies the manual analysis required to replicate the path. This information was used in case study A described in Section 4.4.

If p_t was located in a virtual function successfully resolved during step S4, DroidReach++ will generate the shortest path within the refined ICFG produced at the end of step S4, which only contains the sources that reach the *consumer* method. Often, as is the case in the running example, these will be different sources from the ones that originally executed the *producer* method. All potential paths to reach the producer can be generated by running the reachability analysis with the producer function's entry point as a sink. This distinction can be valuable when determining the sequence of mutually independent method triggers required to reach the vulnerable area of code during manual dynamic analysis, or conversely to identify potential vulnerabilities related to the exploitation of uninitialized or partially initialized data.

Running example. Assuming that N20 is part of a code point related to a known vulnerability in `other-native-lib.so`, DroidReach++ builds the ICFGs in Fig. 7 and computes the path {J8, J9, J10, J11, J12, J16, N9, N10, N11, N17, N18, N19, N20}.

3.9. Discussion

DroidReach++ is designed to statically analyze Android applications and identify execution paths from sources to sinks. Identifying such paths is highly valuable in various scenarios. First, the output produced by DroidReach++ can assist security analysts in discovering potential vulnerabilities by highlighting the control flow leading to critical code points. From a practical perspective, this additional contextual information may allow security analysts to detect complex control flows which may only surface after multiple interactions with the native layer, such as the storage and subsequent usage of native memory pointers into global variables on the Java layer. These include many common and potentially very impactful memory corruption issues in native code, such as references to null pointers or partially uninitialized data, use-after-free and other vulnerabilities caused by improper heap memory management, and time-of-check to time-of-use vulnerabilities. It also may help security analysts to verify whether known vulnerabilities in included shared libraries can be exploited by the primitives offered by the victim app and identified as sources. In many cases, vulnerable portions of larger shared objects, such as cryptographic libraries, can remain unused or are otherwise unexploitable given the limited user influence on the inputs of the vulnerable code path. Second, this output can serve as input to other static or dynamic analysis techniques. For example, symbolic and concolic execution approaches can benefit from guided path information to explore relevant portions of the code more efficiently. Similarly, directed fuzzers can leverage such paths to focus input generation towards exercising specific program behaviors, thereby increasing the likelihood of reaching and testing critical or hard-to-reach code locations. As a result, DroidReach++ can function as a key component within a broader ecosystem of tools working together to enhance the security of Android applications.

4. Experimental evaluation

In this section, we evaluate the efficacy of DroidReach++. We omit the discussion of Step S1 as it involves well-known mainstream Java analysis frameworks, which we did not alter in DroidReach++. The experiments were carried out in a Ubuntu 20.04 Docker container, using two Intel Xeon E5-4610v2 CPUs and 256 GB of RAM and two Intel Xeon Gold 6238R CPUs and 675 GB of RAM. APK hashes of evaluated apps can be found at Borzacchiello (2022a).

4.1. Microbenchmarks

To validate DroidReach++, we considered existing benchmarks from the Android literature. DroidBench (Arzt et al., 2014) does not involve native libraries. On NativeFlowBench (Wei et al., 2018a), DroidReach++

Table 1
Description of the microbenchmarks and experimental results when running Argus-SAF, Ghidra, and DroidReach++, over the microbenchmarks. The source code is available at Borzacchiello (2022b).

Challenge	ID	Description	Reachability challenge solved by		
			Argus-SAF	Ghidra	DroidReach++
C1	0	JNI mapping through static name mangling.	✓	–	✓
C1	1	JNI mapping through static name mangling and method overloading.	✓	–	✓
C1	2	JNI mapping dynamically defined using the `RegisterNatives` API.	✓[a]	–	✓
C1	3	JNI mapping dynamically defined using the `RegisterNatives` API but with *clash* in the class name.	✓[a]	–	✓[a]
C1	4	JNI mapping dynamically defined using the `RegisterNatives` API but without following the Android guidelines.	✓[a]	–	✓[a]
C1	5	JNI mapping dynamically defined using the `RegisterNatives` API with a hard-to-analyze `JNI_OnLoad`.	✗	–	✓
C2	6	JNI Method calls a function from a *nested* library.	✗[b]	✗[c]	✓
C3	7	The target function is called at a *high* calldepth.	✗[d]	✓	✓
C3	8	The target function is called after an indirect call (C++ virtual call, lazy initialization).	✓	✗	✓
C3	9	The target function is called after an indirect call (C++ virtual call, callback).	✓	✗	✓
C3	10	The target function is called after an indirect call (function pointer).	✓	✗	✓
C3	11	The target function is called at a *high* calldepth after an indirect call (*small* calldepth after the indirect call).	✗[d]	✗	✓
C3	12	The target function is called at a *high* calldepth after an indirect call (*high* calldepth after the indirect call).	✗[d]	✗	✗[d]
C4	13	The target function is called after an indirect call (Producer-Consumer pattern, virtual method originating from main class).	✗	✗	✓
C4	14	The target function is called after an indirect call (Producer-Consumer pattern, virtual method originating from inherited class).	✗	✗	✓
C4	15	The target function is called after an indirect call (Producer-Consumer pattern, virtual method from member of main class).	✗	✗	✗[d]

✓: the tool is able to solve the reachability challenge.
✗: the tool is unable to solve the reachability challenge.
–: the tool is not designed for solving the reachability challenge.
[a] The tool exploits symbolic execution to solve the reachability challenge.
[b] Our improved version of Argus-SAF, dubbed Argus-SAF-MLIB, can solve this reachability challenge.
[c] Our improved version of Ghidra can solve this reachability challenge.
[d] While the analysis calldepth can be increased, there will always be a trade-off between running time/memory usage.

Table 2
Statistics for the apps selected for the evaluation.

Downloads range	# apps	Avg # Java insns.	Avg # Java `native` methods	Avg native insns	Avg # ARMv7 libs
100K–1M	32	1,228,452	212.81	2,308,222	18
1M–10M	89	1,533,235	229.24	2,654,673	10.42
10M–100M	132	1,849,999	204.86	2,554,607	7.52
100M–500M	201	1,515,141	209.48	2,841,205	7.81
500M–1B	28	1,649,847	235.43	2,511,424	9.82
1B+	18	1,945,265	565.94	2,392,670	5.39

performs consistently with Argus-SAF when considering the reachability goal. Since NativeFlowBench ignores the challenges from Section 3.2, we designed a new benchmark suite composed of 16 apps that exhibit these aspects from different perspectives (see Table 1). The source code and a detailed discussion of this suite can be found in a dedicated repository (Borzacchiello, 2022b). On 14 out of 16 apps, DroidReach++ correctly builds accurate native ICFGs, improving on Argus-SAF (8 out of 16). The failing case in benchmark #12 involves an indirect jump at *deep* call depth: this app was specifically designed to highlight that DroidReach++ cannot solve, in general, the scalability issues that inherently affect static analyses, and it can only help to mitigate them (hopefully in several cases). Although DroidReach++ can be configured to run its analysis at higher depths, the default nested method depth of 5 is considered a good compromise between covering as many usages of native library recursion as possible and containing computing time. More details on the results are available at Borzacchiello (2022b).

Compared to the initial release of our benchmark (Borzacchiello et al., 2022), in this extension, we added three additional benchmarks to evaluate the *producer–consumer patterns* emerging in context-based executions. These are patterns that none of the other comparable tools resolve correctly, and works as a showcase of DroidReach++'s current capabilities. Correctly resolving the ICFG for these scenarios can be extremely complex, as discussed in the previous sections, and may require significantly more computing time compared to all other patterns. However, in most situations, DroidReach++ is capable of providing at least partial information on how the native object produced by one native method is utilized by its consumer. More specifically, benchmark #13 implements a virtual function call, jumping into a method originally defined or overridden by the main class, benchmark #14 implements a virtual function call to a method inherited by an unknown ancestor, and benchmark #15 implements a virtual function call to a method contained in a member of the main class. With the term *main class*, we indicate the native object that is instantiated by the producer method. The virtual methods utilized by the consumer in benchmarks #13 and #14 are stored in the virtual table of the main class, and are successfully extracted and integrated into the call graph (and successively into the full ICFG), as described in Section 3.7. Any virtual function call or pointer access at a higher depth, such as those from a dereferenced class member as in benchmark #15, cannot be automatically resolved but can still be *detected* due to the jump through its control flow into a memory area we provided as part of the initial setup. The consumer is then mapped to the most likely producer,

Table 3
Analysis of false negatives: executed code points found in the ICFGs.

APK	# of executed code points found in the native ICFGs from		
	Argus-SAF	Ghidra	DroidReach++
com.sec.android.easyMover	33/33	33/33	33/33
com.jb.zcamera	11/11	11/11	11/11
com.mi.android.globalFileexp.	31/48	47/48	47/48
com.space.cleaner.smart.tool	52/75	65/75	75/75
com.soundcloud.android	57/102	72/102	72/102
video.like	197/518	272/518	320/518
com.zentertain.photocollage	186/331	174/331	239/331
com.picsart.studio	442/1282	203/1282	736/1282
shareit.lite	33/60	44/60	47/60
com.imangi.templerun	60/326	67/326	248/326
com.amazon.mp3	92/395	218/395	239/395
com.cam001.selfie	282/344	211/344	322/344
com.tripadvisor.tripadvisor	2/42	30/42	30/42
com.yodo1.crossyroad	3/79	25/79	25/79
com.king.candycrushjellysaga	5/54	6/54	6/54

Table 4
Analysis of false positives: validated code points in the ICFGs.

APK	Validation mode	Confidence	# code points to validate	# validated code points
com.imangi.templerun	Dynamic	High	2357	1565
com.picsart.studio	Dynamic	High	1307	748
com.cam001.selfie	Dynamic	High	974	326
com.king.candycrushjellysaga	Dynamic	High	168	63
com.amazon.mp3	Mixed	Medium	625	441
shareit.lite	Static	Medium	12	12
com.sec.android.easyMover	Static	Medium	107	60

following the heuristics described in Section 3.7. This has the benefit of still offering the partial ICFG up to the virtual method call, allowing us to stimulate that code point during dynamic analysis.

4.2. Real-world dataset

Dataset. To evaluate the efficacy of DroidReach++ on real-world apps, we collected the top-20 apps — according to their popularity — (minimum 100 K downloads) from each category and sub-category (e.g., the games category includes arcade, sports, etc.) of the Google Play Store, keeping the ones containing ARMv7 libraries. Overall, we obtained 500 apps organized in three macro-categories: (i) 184 game apps; (ii) 236 apps belonging to productivity, browsing, education and multimedia; (iii) 80 apps belonging to other categories. Such a selection choice has also been guided by the idea of representing apps whose vulnerability may have a very large *impact* on the end-users. The average *complexity* of these apps is very high, as detailed in Table 2, in terms of the number of Java and native instructions (more than 2.3 million of native LoC on average), the number of Java `native` methods (more than 204 methods on average) and of ARMv7 libraries (at least 5 on average).

Fine-grained evaluation. To evaluate the correctness of Droid Reach++, we need to analyze false negatives (code points missing from the ICFGs) and false positives (code points that are wrongly inserted in the ICFGs). In particular, such practical evaluation allows us to ascertain *without doubts* that a code point can be effectively reached. However, this analysis also features various challenges: as shown in previous works, a thorough dynamic evaluation of an Android app would require exploring *every single code branch*, an often unfeasible procedure, especially for very complex applications. For this reason, we decided to stimulate the tested applications through *direct user interactions*, i.e., by analyzing the code points reached when the application is *normally used*. In this way, we can provide results with a much greater practical impact. However, this is a very time-consuming operation that limits the number of applications that can be tested.

Concerning false negatives, we randomly selected 15 apps from our dataset and then manually stimulated them in the Android emulator as a user would do in a short usage session, recording the executed native function entry points. Thus, any recorded code point should be contained in the ICFGs. Although our sample set may seem small, the effort for validating the results took more than 1.5 man-months. Table 3 divides the 15 apps into three groups: applications where DroidReach++ was able to identify more than 95% of the executed code points are in the first group, more than 50% of the code points in the second group, and less than 50% of code points in the third group, respectively.

DroidReach++ significantly outperforms Argus-SAF and Ghidra (when used in step S3 instead of our tool) in several apps. This result comes from the effective combination of different techniques: Argus-SAF fails to scale its analysis and Ghidra misses indirect jumps that could be recovered with a symbolic execution analysis, while DroidReach++ shows the best of the two approaches (see Borzacchiello, 2022a for detailed debug results). However, there are some apps where even DroidReach++ is unable to statically recover some executed code points. In our experiments, we set the maximum call depth to 5. This choice was guided by empirical observations during the evaluation of the analyzed apps, where this value offered the best trade-off between accuracy and analysis time. Furthermore, a similar setting was used in previous state-of-the-art approaches, ensuring consistency with prior experimental setups during comparison. However, in some apps, slightly increasing the maximum call depth in symbolic exploration can lead to better results (e.g., from 5 to 10 allows to find +9% of executed code points in `com.amazon.mp3`). Similarly, increasing the maximum analysis timeout can improve the accuracy, but there is a trade-off that must be taken into account between accuracy and analysis time. Even when extending the analysis time, DroidReach++ cannot cope with some patterns (Section 5): for example, `com.yodo1.crossyroad` loads a library using a custom loader and `com.imangi.templerun` indirectly executes code from the Mono framework.

Concerning false positives, the evaluation is significantly harder as it requires to *exhaustively* stimulate an app, which can hardly be done *automatically* for most apps. However, we attempted to validate at least

Table 5
Resolved JNI mappings during step S2.

	# Recovered mappings		Analysis time (s)	
	Argus-SAF	DroidReach++	Argus-SAF	DroidReach++
Static JNI mappings	4610	4610	8542	8542
Dynamic JNI mappings	765	1912	259,136	20,345
Both	5375	6522	267,678	28,887

Table 6
ICFG results on methods analyzed using different approaches in S3.

DroidReach++ vs	# JNI methods processed by both	% apps for which DroidReach++ has less code points than competitor	same	more	# code points found by DroidReach++ (ratio w.r.t. competitor) Total	Total ratio	geo. mean ratio
Ghidra	5623	2.8%	9.8%	87.3%	64,818,031	1.24×	1.95×
Argus-SAF	4711	1.3%	7.2%	91.5%	54,901,175	7.58×	5.64×
Argus-SAF-MLIB	4527	1.6%	8.5%	89.9%	51,618,223	6.51×	5.09×

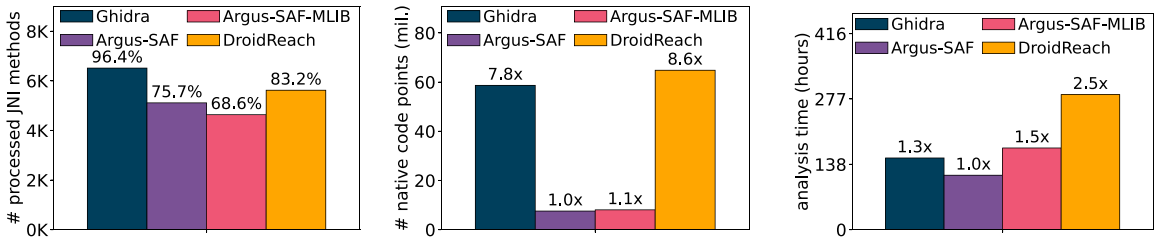

Fig. 8. ICFG results (step S3) on the full set of JNI methods over all apps.

a subset of the code points. In particular, we compared the ICFGs from DroidReach++ with those from Ghidra and Argus-SAF, extracting the code points detected only by our approach and then keeping only the function entry points. To keep the evaluation sustainable, we considered a subset of the 15 apps and analyzed how to stimulate their JNI methods based on the reports from FlowDroid (S1). We then ran each app on a debugger during an extended usage session, tracking which function entry points found by DroidReach++ were actually executed. Table 4 shows the results of our experiments. In four applications, we were able to validate a large fraction of the selected code points, bringing high confidence in their correctness.

In other three applications, we could not dynamically validate most code points. This is not unexpected, as several program behaviors depend on external events (e.g., server-side interactions) and specific usage patterns that cannot always be reproduced. For example, the code points considered from `com.sec.android.easyMover` are within a library related to the USB OTG functionalities on Samsung devices, which we could not stimulate. In `shareit.lite`, the code points are mostly related to C++ exception handling, making them hard to trigger. In `com.amazon.mp3`, we experienced some crashes when inserting the breakpoints into some libraries, allowing us to dynamically validate only a few points. For these three applications, we also performed a *manual* static validation by analyzing a subset of their code points with IDA Pro, validating whether the points are *reasonable*, i.e., they are likely reachable within an execution. However, we report lower confidence as we did not validate them by running the app. Regarding unvalidated code points, they should not necessarily *all* be seen as false positives, as proving or disproving their correctness is hard: static analyses are often proposed when automatic dynamic analyses cannot exhaustively cover the program code. We provide additional details in Borzacchiello (2022a). Overall, the effort for this validation was more than 1.5 man-months.

Coarse-grained evaluation. To get a wider evaluation of Droid Reach++, we now consider the full dataset. We focus our discussion on steps S2 and S3. Step S1 brings the same results for all tools as they can use the same Java analysis framework. Similarly, step S5 can be implemented in the same way for all compared tools. In contrast, step S4 is conceived only by DroidReach++ and thus is discussed in isolation only for our framework in Section 4.3.

A crucial challenge addressed by DroidReach++ during step S2 is to identify mappings between Java native methods and JNI methods. In our dataset, step S1 identifies 7463 *reachable* Java `native` methods. This is quite interesting since the total number of Java `native` methods in our dataset is 113,316: this suggests that even if an app contains some code, it may not necessarily execute it. Our manual investigation has confirmed that most apps integrate third-party frameworks, but often only use a subset of their functionalities.

Table 5 reports the cumulative number of mappings successfully resolved by DroidReach++ compared to Argus-SAF, and the cumulative analysis time for these two approaches. Overall, a large fraction (61.8% of 7463) of the mappings are statically defined by the apps and can be resolved by both frameworks. The remaining 2853 (38.2%) mappings are defined dynamically though `RegisterNatives`: DroidReach++ performs significantly better than Argus-SAF on these methods, resolving 2.5× dynamic mappings. DroidReach++ is also significantly more efficient: the analysis time is reduced by a factor of 9.3×.

Overall, DroidReach++ has resolved 87.4% of the methods (compared to 72.0% in Argus-SAF), suggesting that 941 (12.6%) methods do not follow the implementation patterns expected by DroidReach++. While the symbolic exploration helped resolve 79 methods, it still failed on the 941 unresolved methods. In these cases, the `JNI_OnLoad` function was too complex and the exploration was aborted after a 15 min timeout. Users can customize this timeout in DroidReach++ to possibly increase the accuracy of S2. While there are still some unresolved mappings, the improvement from DroidReach++ can be quite significant in practice: when considering the Adobe PDF reader (`com.adobe.reader`), all mappings were found exclusively by DroidReach++, meaning that Argus-SAF would completely skip any analysis on the native layer for this app.

After finding the JNI mappings for our dataset, we evaluate the effectiveness and performance of DroidReach++ during the ICFG construction (step S3). Besides our framework DroidReach++, we consider: (a) Ghidra, as it is internally used by DroidReach++, (b) Argus-SAF, which is the main competitor, and (c) Argus-SAF-MLIB, a variant of

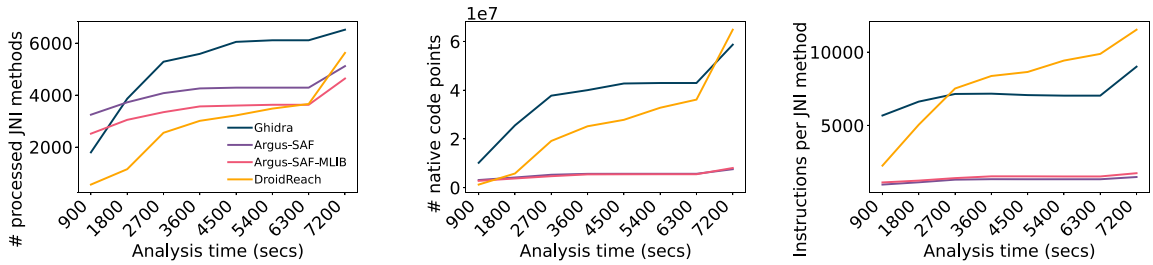

Fig. 9. ICFG results (step S3) on the full set of JNI methods over all apps when considering different analysis timeouts.

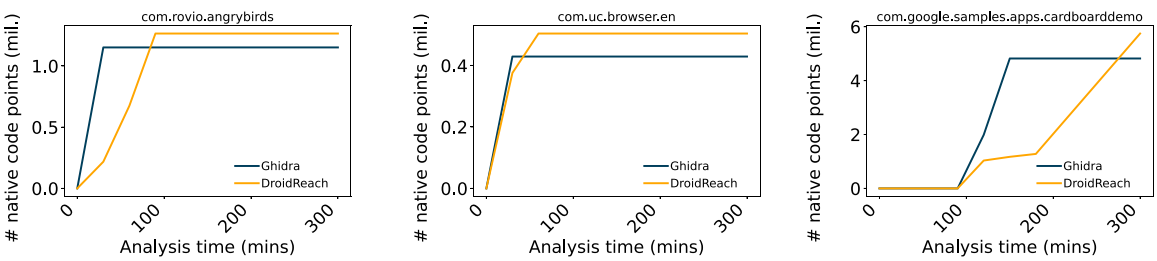

Fig. 10. # code points in the ICFGs (step S3) over time when analyzing three (randomly chosen) applications.

Argus-SAF that we developed, which can continue its analysis even in the presence of nested libraries (while the original approach would ignore them). This is important since 340 (68%) apps out of 500, have at least one nested library and some apps may even have a *nested chain* with up to three libraries. Each solution was executed for 2 h for each application, reconstructing in sequence the ICFGs of the reachable JNI methods. This threshold was chosen empirically, as it represents a reasonable amount of time given the complexity of analyzing large applications. Notably, reducing this time may increase the risk of losing valuable information, while extending it could render large-scale analysis unsustainable without necessarily providing significant benefits. Overall, although defining an ideal threshold is challenging, we believe this choice realistically reflects the characteristics of the applications. To make a fair comparison, all tools received the same output from step S2.

Since different tools come with different trade-offs in terms of accuracy and performance, leading to a very different number of JNI methods processed within the 2-h experiment, we first present in Table 6 a pairwise comparison between DroidReach++ and the other solutions considering the common set of JNI methods which were processed by each pair of frameworks. When considering the 5623 JNI methods analyzed by both DroidReach++ and Ghidra, DroidReach++ can identify more code points in 87% of the apps. On average for each app, DroidReach++ finds 1.95× code points than Ghidra. When considering the 4711 JNI methods analyzed by both DroidReach++ and Argus-SAF, our approach identifies more code points in 91% of the apps. On average for each app, DroidReach++ finds 5.64× code points than Argus-SAF. When considering our custom variant Argus-SAF-MLIB, DroidReach++ is still more effective.

Fig. 8 summarizes the results when considering all JNI methods from all apps: one approach could be less accurate but more efficient on one method, thus having the opportunity to process more methods within the 2-h per-app timeout. The left graph shows that Ghidra was able to process more methods than the competitors, followed by DroidReach++. The right graph confirms that DroidReach++ is indeed slower than Ghidra. However, the center graph shows that the number of code points is still in favor of DroidReach++. This is expected: DroidReach++ performs the same work as Ghidra, plus additional analyses. Hence, its running time is always longer than Ghidra, leading to some apps reaching the 2-h per-app timeout before processing the entire set of methods.

When comparing DroidReach++ to Argus-SAF and Argus-SAF-MLIB, the results in Fig. 8 show that DroidReach++ was able to process more methods than these two solutions, detecting ~8× their number of code points but requiring also a larger analysis time. Indeed, Argus-SAF (and Argus-SAF-MLIB) are generally faster (−60%) than Ghidra (and thus DroidReach++) for a large set (~60%) of methods but: (a) these solutions are significantly less accurate, identifying fewer code points on this large set, and (b) on the other methods, these solutions fail to generate any ICFG as they reach the timeout or saturate the memory very early (25 GB in our experiments) due to path explosion. When attempting to increase the maximum call depth in Argus-SAF-MLIB, we observed a crucial increase in the number of timeouts and out-of-memory events.

Finally, the average analysis time of DroidReach++ per app was 0.7 h, 0.4 for Ghidra, 0.3 for Argus-SAF, and 0.4 for Argus-SAF-MLIB. To obtain a more fine-grained evaluation of DroidReach++'s scalability, Fig. 9 shows the performance of the different approaches when running the analysis in parallel across all apps, while varying the analysis timeout. As previously suggested by Fig. 8, DroidReach++ is slower than Ghidra in processing JNI methods. However, it extracts more code points, resulting in richer ICFGs (in terms of the number of instructions). Compared to Argus-SAF and Argus-SAF-MLIB, DroidReach++ is more effective overall, although it may take longer initially to reach the same number of processed JNI methods. These findings are further supported by Fig. 10, which shows the number of code points in the ICFGs over time for three randomly selected apps from our dataset (`com.rovio.angrybirds`, `com.uc.browser.en`, and `com.google.samples.apps.cardboarddemo`). The analysis timeout was set to 5 h, allowing each tool sufficient time to complete the analysis. For 2 out of 3 apps, Ghidra completes its analysis in just 30 min, while DroidReach++ takes up to 90 min. However, at the end of the analysis, DroidReach++ produces richer ICFGs in terms of code points. In the third app, both tools are significantly slower: Ghidra takes about 150 min, and DroidReach++ requires nearly the full 5 h. Despite this, DroidReach++ ultimately proves more effective than Ghidra in identifying a greater number of code points.

4.3. Context-Based JNI executions

Concerning the analysis of Context-Based JNI execution reported in Section 3.7, we devised an experiment aimed at exploring the

Table 7
Consumer-producer analysis (step S4).

	DroidReach++	Angr
% apps with at least one proved consumer-producer	5.4%	1.3%
% JNI methods that are proved consumers out of 1794 potential consumers	4.8%	1.4%
Average analysis time (secs) for each potential consumer	207.7	370.3

Table 8
Improvement during ICFG construction due to step S4 for JNI methods that have been proved consumers.

	# code points	Total ratio	geo. mean ratio
DroidReach++ w/ S4	413,131	1.47×	3.44×
DroidReach++ w/o S4	280,684		

capabilities of DroidReach++ to discover producer–consumer paths. Specifically, we analyzed for 2 h each app from our dataset, looking for consumers and producers using step S4 of DroidReach++ (using a 5 min timeout for each path emulation). Since Argus-SAF does not consider challenge C4, we report what a plain strategy based on symbolic execution (dubbed Angr) may perform when aimed at detecting consumer-producer pairs.

As reported by Table 7, DroidReach++ identifies at least one valid pair of producer–consumer in 5.4% of the apps. When considering the 1794 JNI methods receiving at least one `long` argument, DroidReach++ detects that 4.8% of them are object consumers, successfully finding also their object producer. On the other hand, the strategy based on symbolic execution performs significantly worse, finding a valid pair of producer–consumer in 1.3% of the apps and 1.4% of the JNI methods.

The results of DroidReach++ derive from improved scalability: the average analysis time per *potential* object consumer was 207.7 s in DroidReach++ compared to the 370.3 s of the symbolic exploration. This efficiency is an especially important point when considering the bias of this pattern's prevalence in highly complex applications: while we successfully match a native function to a Java native method call at least once in 35.9% of all applications, those applications contain 45.5% of all native instructions and 65.7% of all methods.

After enabling the consumer tagging option, our framework Droid Reach++ selects 8.5% of the 1794 methods as consumers, corresponding to 8.8% of total apps, although 38.6% of these methods could not be matched to a producer. This is most likely due to the much higher level of complexity captured by this pattern compared to the main system, producing incomplete results within the time limit even with these efficiency improvements.

Table 8 shows the improvements during ICFG reconstruction thanks to step S4. The total number of code points for the object consumers increases by 1.47×, with an average improvement per app of 3.44×. For instance, DroidReach++ is able to find 13.4× code points in the object consumers from the Opera browser (`com.opera.browser`) when enabling step S4. Another positive example is the live stream platform Twitch (`tv.twitch.android.app`), where several JNI methods from the `CoreAPI` library follow the producer–consumer pattern (2.2× code points when enabling S4). The results on these two apps were manually validated to verify their correctness. The Google Meet app (`com.google.android.apps.tachyon`) makes extensive use of this pattern, and consequently we could obtain a massive improvement in its ICFG, going from 7 to 49 653 code points integrated into the final reachable function graphs. Adobe Reader (`com.adobe.reader`) shows poor results with Argus-SAF due to a proprietary rendering engine which adopts the producer–consumer, thus seeing an increase of 1.5× in total code points on DroidReach++ after enabling S4 in DroidReach++.

More in general, the improvements are most noticeable in apps built around wrappers for game engines and web browsers or image rendering engines, which can often contain native code not specifically targeted to Android, and as such its use in deeply nested functions could go undetected in normal circumstances. For instance, the Samsung Internet Browser (`com.sec.android.app.sbrowser`), which is built on Chromium similarly to Opera, went from 0 to 12 consumer matches out of the 84 potential native methods, of which 4 were successfully paired with a producer. When manually validating these pairs, we are confident in the correctness of 3 out of 4, while the fourth may have different producers depending on the application's execution flow. As for the remaining 8 identified consumers, they unfortunately cannot be matched with a valid producer due to the maximum depth limitation imposed by the emulation in Algorithm 6. While this constraint improves scalability, it can, as in this case, result in false negatives.

Similarly, the apps `google.free.icequeen. peachygames-llc` and `com.super_toki.gi- raffe`, which utilize the Corona/Solar2D game engine, went from 1 to 7 and 6 consumer-producer pairs, respectively. When reviewing the pairs identified in `com.super_toki.giraffe`, we manually validated their correctness in full. Regarding false negatives, static analysis of the native libraries reveals several JNI methods that could potentially act as consumers, as they likely take a pointer to a C++ object. However, these methods are not marked as reachable during S1. Therefore, assuming that S1 produces correct results, our analysis appears to behave as expected. Finally, other patterns already discussed, such as the Mono framework most commonly used in the Unity game engine, are still not resolved by this approach. However, we believe that our approach represents a significant improvement for many code bases centered around simple multi-platform wrappers, which often only have one level of nesting between the JNI function call and the main native object, handling function signature changes and other type conversions. This, concurrently with the support for multi-library references, is also the main reason for DroidReach++'s much stronger performance in Chromium-based apps.

Worth noting, the producer–consumer detection capabilities (exhibited by DroidReach++ as a whole) represent a significant improvement in comparison to previous state of the art tools. This functionality can be particularly useful to detect vulnerabilities yielded by the producer–consumer interaction, which is not accurately modeled by any of the analysis tools tested.

4.4. Case studies

Establishing that apps contain vulnerable libraries *does not mean* that such functions necessarily constitute an *immediate* security concern. We present two case studies where DroidReach++ can be used as an aid in evaluating the *impact* of vulnerable functions. These apps were considered by a previous study (Almanee et al., 2021; Almanee, 2021).

Case study A: reachable function. We consider the function `BN_bn2dec` from `libcrypto.so`. This function is used in Amazon Alexa (`com.amazon.dee. app`) and is vulnerable in OpenSSL ≤ 1.1.0 (CVE-2016-2182 (CVE, 2016)) with a score of 7.5. DroidReach++ finds the following path (depicted in Fig. 11):

- The Java layer loads the `OnStartCommand` function belonging to the `com.here. android.mpa.service.MapSer- vice` class. This function loads the (name obfuscated) `a` method from the `com.nokia.maps.SSLCertManager` class.
- This method calls the `x509_NAME_HASH` `native` function that belongs to the `com.nokia.maps.CryptUtils` class, which is statically mapped to the JNI method `com_nokia_maps _CryptUtils_X509_1NAME_1HASH` in `libMA- PSJNI.so`.

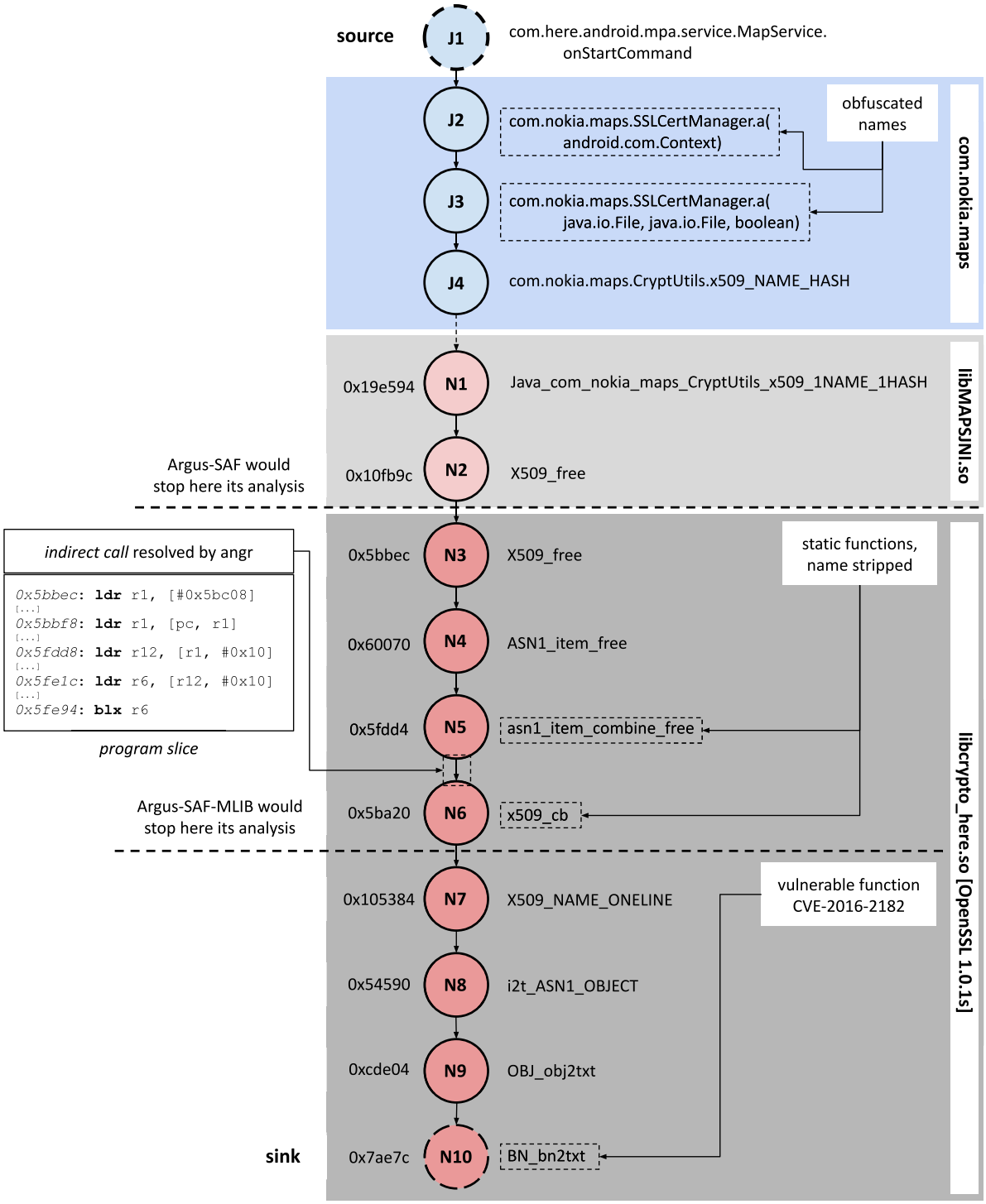

Fig. 11. Path found in the Amazon Alexa app (`com.amazon.dee.app`) that can reach the vulnerable function `BN_bn2txt` from OpenSSL (CVE-2016-2182 CVE, 2016).

- The JNI method calls `X509_free`, which is a function from `libcrypto_here. so`, that in turn invokes `ASN1_item_free`, which calls a stripped function at offset `0x5fdd4` (after reversing, it appears to be `asn1_item_-combine_free`).
- From this function, the static exploration becomes challenging. There are no direct jumps that connect the function to the target sink. However, DroidReach++ identifies a reachable offset `0x5ba20` (which would allow for further exploration towards the sink). A deeper inspection shows that such an offset is *indirectly* calculated and jumped to by accessing dedicated data structures. This is the reason why the connection between the offsets was not immediately evident. Moreover, it demonstrates the capability of Droid Reach to identify non-obvious paths that do not involve direct jumps.
- The function at offset `0x5ba20` calls `X509_NAME_ONELINE`, which invokes `i2t_ ASN1_OBJECT`. Such a function invokes `OBJ_obj2txt`, which calls `BN_bn2dec`.

After having statically identified a path, we tried to stimulate it dynamically. Unfortunately, reproducing it in the emulator is not easy: besides registering an account and performing several interactions, additional

events must be faked to execute the interesting Java class. Nonetheless, we successfully reproduced a similar path in `com.nokia.maps`, which includes the same third-party library. Argus-SAF and Ghidra miss some crucial edges, failing to find the path.

Case study B: unreachable function. The goal of this case study is to ascertain whether there is *no path* to a target vulnerable function. We consider Zoom (`us.zoom.video meetings`) and the function `SRP_VBASE_get_by_user` in `lib- crypto.so` (CVE-2016-0798, score 7.8), for which our tool could *not* find a path. To validate our claim, we directly patched the native library function with an interrupt `svc 11` instruction to see whether the function was invoked. Then, we tested all possible functionalities. The application showed no signs of a crash, meaning that the target function was not invoked during the execution. Although we *cannot* guarantee that the function will *never* be invoked, we believe that it cannot be executed by a normal user under normal conditions.

5. Limitations

Our current implementation of DroidReach++ has a few limitations:

- DroidReach++ was not designed as a vulnerability detector, as that would require generating the proper inputs to trigger undefined behaviors on the applications. Notably, this is a challenging task that demands separate tools and methodologies, as described in Section 2. However, DroidReach++ represents a fundamental step in this direction. Understanding that a function could be effectively reached helps analysts select which vulnerability to address and stimulate. Hence, the role of reachability analysis is highly strategical in the context of vulnerability assessment.
- Like Argus-SAF, DroidReach++ is currently tuned for ARMv7 code. However, from the methodological side, nothing is tied to a specific architecture.
- DroidReach++ looks for native libraries in standard locations: fixes may be needed in the case of a custom loader or packed libraries.
- While DroidReach++ considers all methods connected to essential user interactions as path sources, as described in Section 3.4, it cannot prove the feasibility of a path, i.e., it does not currently generate the precise *inputs* or *stimulations* that can reproduce the execution path. Unfortunately, existing static solutions (Wei et al., 2018a) do not scale on large apps.
- DroidReach++ represents the structure of the code using ICFGs. This representation may be inadequate in the presence of frameworks that deviate significantly from the traditional Android programming environment.
- While DroidReach++ is the only tested tool capable of following the producer–consumer pattern, the support for native objects of higher complexity is still partial. For example, some applications use object constructors based on complex C++ hierarchies involving, for instance, multiple inheritance. In such cases, DroidReach++ may not correctly identify the producer–consumer pattern. This is because the emulation in Algorithm 6 is bounded to a maximum depth to ensure scalability, which may result in incomplete tracking of virtual tables. Consequently, this can lead to virtual table validation failures when analyzing consumers. Another source of false negatives arises from the runtime loading of Java and native code. Although this practice violates Android guidelines and terms of service, it still occurs in the wild. When this happens, DroidReach++ may be unable to detect the native code loaded at runtime. Lastly, runtime environments such as Mono .NET can disrupt reachability analysis, potentially resulting in false negatives when determining the candidate set of producers and consumers. Finally, DroidReach++ only considers JNI methods that are marked as reachable during phase S1 as potential producers and consumers. Since S1 relies on existing state-of-the-art solutions that we did not modify, our analysis is inherently dependent on their accuracy.
- The analysis performed by DroidReach++ is currently serialized when considering an app, as the approach processes one JNI method at a time. To improve scalability, the implementation could be refactored to parallelize the analysis across JNI methods. However, some dependencies of DroidReach++, such as Ghidra, may not be thread-safe, potentially requiring the use of separate process instances for concurrent analyses. Additionally, our approach would benefit from a more sophisticated refinement mechanism to handle cases where the ongoing analysis of one JNI method uncovers code points relevant to other JNI methods currently under analysis.
- Our false negative analysis was conducted on 15 apps, a relatively small subset of the original 500. As mentioned in Section 4.2, the evaluation of these apps took more than 1.5 man-months. Future work may focus on expanding such analysis in a more automatic fashion. One possibility involves leveraging the debugging and scripting capabilities of professional disassemblers like IDA and Ghidra. The idea would be constructing a pipeline where the application is automatically stimulated by using automatic/guided UI interaction (e.g., Monkey Android Developers, 2021) and by automatically placing specific breakpoints in native calls.
- DroidReach++ focuses strictly on the analysis of shared libraries bundled with the app package. This approach allows us to focus on scenarios where both app developers and security researchers can evaluate the impact of newly discovered vulnerabilities within app dependencies. Future developments may extend this analysis to any native library loaded and interacted with by the Java layer, including Android system libraries, in order to detect potentially malicious behavior in app samples.

6. Conclusions

This paper presented DroidReach++, a novel analysis tool for Android apps to assess the reachability of native functions. Specifically, we used DroidReach++ to conduct an extensive static analysis of 500 real-world Android apps to solve several reachability challenges. The extension of our previous work focused in particular on understanding context-based native execution for applications embedding C++ code. Our experiments show that DroidReach++ can reconstruct more accurate ICFGs for several popular apps with respect to state-of-the-art solutions. Our two case studies show how security experts can use DroidReach++ to assess whether a vulnerable function could be reached during the execution, providing evidence whether a vulnerability, e.g., inside a third-party library, constitutes an immediate risk for the users. Moreover, the results provided by DroidReach++ depict intriguing scenarios in which function reachability still constitutes a major technical challenge to address and validate. Future research works will especially concern automatic validation of results in relation to the feasibility of paths.

CRediT authorship contribution statement

Luca Borzacchiello: Writing – original draft, Software, Methodology, Investigation, Data curation, Conceptualization. **Matteo Cornacchia:** Software, Methodology, Investigation, Data curation, Conceptualization. **Davide Maiorca:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Funding acquisition, Conceptualization. **Giorgio Giacinto:** Validation, Supervision, Funding acquisition, Conceptualization. **Emilio Coppa:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Matteo Cornacchia, Davide Maiorca, Giorgio Giacinto, and Emilio Coppa reports financial support was provided by Italian Ministry of Education, Universities and Research (MUR), European Union - NextGenerationEU, Italian National Cybersecurity Agency (ACN). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was partially supported:
• Project SERICS 1527 (PE00000014) under the NRRP MUR program funded by the EU-1528 GEU.
• Project FARE (PNRR M4.C2.1.1 PRIN 2022, Cod. 202225BZJC, CUP D53D23008380006, Avviso D.D 104 02.02.2022) under the Italian NRRP MUR program funded by the European Union - NextGenerationEU.
• Project SETA (PNRR M4.C2.1.1 PRIN 2022 PNRR, Cod. P202233M9Z, CUP F53D23009120001, Avviso D.D 1409 14.09.2022) under the Italian NRRP MUR program funded by the European Union - NextGenerationEU.
• Agenzia per la cybersicurezza nazionale under the programme for promotion of XL cycle PhD research in cybersecurity – CUP B83C24005 670005.

The views expressed are those of the authors and do not represent the funding institutions.

Data availability

The code is publicly available on Github. The hashes of the Android APKs used for the experiments is available on the GitHub repository.

References

Abraham, A., Andriatsimandefitra, R., Brunelat, A., Lalande, J., Tong, V.V.T., 2015. GroddDroid: a gorilla for triggering malicious behaviors. In: 10th Int. Conf. on Malicious and Unwanted Software. MALWARE '15, http://dx.doi.org/10.1109/MALWARE.2015.7413692.

Almanee, S., 2021. Librarian dataset. URL: https://github.com/salmanee/Librarian.

Almanee, S., Ünal, A., Payer, M., Garcia, J., 2021. Too quiet in the library: An empirical study of security updates in Android apps' native code. In: 43rd IEEE/ACM Int. Conf. on Software Engineering. ICSE '21, http://dx.doi.org/10.1109/ICSE43902.2021.00122.

Amalfitano, D., Fasolino, A.R., Tramontana, P., De Carmine, S., Memon, A.M., 2012. Using GUI ripping for automated testing of android applications. In: Proceedings of the 27th IEEE/ACM Int. Conf. on Automated Software Engineering. ASE '12, http://dx.doi.org/10.1145/2351676.2351717.

Android, 2021. Native libraries. URL: https://developer.android.com/training/articles/perf-jni#native-libraries.

Android Developers, 2021. UI/Application exerciser monkey.

Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Octeau, D., McDaniel, P., 2014. FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In: Proc. of the 35th ACM SIGPLAN Conf. on Programming Language Design and Implementation. http://dx.doi.org/10.1145/2594291.2594299.

Bello, L., Pistoia, M., 2018. ARES: Triggering payload of evasive Android malware. In: Proc. of the 5th Int. Conf. on Mobile Software Engineering and Systems. MOBILESoft '18, http://dx.doi.org/10.1145/3197231.3197239.

Borzacchiello, L., 2022a. DroidReach. URL: https://github.com/season-lab/DroidReach.

Borzacchiello, L., 2022b. DroidReach Benchmarks. URL: https://github.com/season-lab/DroidReachBenchmarks.

Borzacchiello, L., Coppa, E., Demetrescu, C., 2021a. Fuzzing symbolic expressions. In: Proc. of the 43rd Int. Conf. on Soft. Eng.. ICSE '21, http://dx.doi.org/10.1109/ICSE43902.2021.00071.

Borzacchiello, L., Coppa, E., Demetrescu, C., 2021b. FUZZOLIC: mixing fuzzing and concolic execution. Comput. Secur. http://dx.doi.org/10.1016/j.cose.2021.102368.

Borzacchiello, L., Coppa, E., Maiorca, D., Columbu, A., Demetrescu, C., Giacinto, G., 2022. Reach me if you can: On native vulnerability reachability in android apps. In: Atluri, V., Pietro, R.D., Jensen, C.D., Meng, W. (Eds.), Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part III. In: Lecture Notes in Computer Science, vol. 13556, Springer, pp. 701–722. http://dx.doi.org/10.1007/978-3-031-17143-7_34.

Bosu, A., Liu, F., Yao, D.D., Wang, G., 2017. Collusive data leak and more: Large-scale threat analysis of inter-app communications. In: Proc. of the 2017 ACM on Asia Conf. on Computer and Communications Security. http://dx.doi.org/10.1145/3052973.3053004.

Chen, T., Zhang, X.-S., Guo, S.-Z., Li, H.-Y., Wu, Y., 2013. State of the art: Dynamic symbolic execution for automated test generation. Future Gener. Comput. Syst. http://dx.doi.org/10.1016/j.future.2012.02.006.

Choi, W., Necula, G., Sen, K., 2013. Guided GUI testing of android apps with minimal restart and approximate learning. In: Proc. of the 2013 ACM SIGPLAN Int. Conf. on Object Oriented Programming Systems Languages & Applications. OOPSLA '13, http://dx.doi.org/10.1145/2509136.2509552.

CVE, 2016. CVE-2016–2182. URL: https://www.cvedetails.com/cve/CVE-2016-2182/.

Derr, E., Bugiel, S., Fahl, S., Acar, Y., Backes, M., 2017. Keep me updated: An empirical study of third-party library updatability on Android. In: Proc. of the 2017 ACM SIGSAC Conf. on Computer and Communications Security. CCS '17, http://dx.doi.org/10.1145/3133956.3134059.

Desnos, A., 2021. Androguard. URL: https://github.com/androguard/androguard.

Documentation, A., 2021. Pointer to a native structure in a java field. URL: https://developer.android.com/training/articles/perf-jni#64-bit-considerations.

Enck, W., Gilbert, P., Han, S., Tendulkar, V., Chun, B.-G., Cox, L.P., Jung, J., McDaniel, P., Sheth, A.N., 2014. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones. ACM Trans. Comput. Syst. http://dx.doi.org/10.1145/2619091.

Eschweiler, S., Yakdan, K., Gerhards-Padilla, E., 2016. discovRE: Efficient cross-architecture identification of bugs in binary code. In: 23rd Annual Network and Distr. Sys. Sec. Symp.. http://dx.doi.org/10.14722/ndss.2016.23185.

Gao, J., Yang, X., Fu, Y., Jiang, Y., Shi, H., Sun, J., 2018. VulSeeker-pro: Enhanced semantic learning based binary vulnerability seeker with emulation. In: Proc. of the 2018 Eur. Soft. Eng. Conf. and Symp. on the Foundations of Soft. Eng.. In: ESEC/FSE 2018, http://dx.doi.org/10.1145/3236024.3275524.

Gordon, M.I., Kim, D., Perkins, J., Gilham, L., Nguyen, N., Rinard, M., 2015. Information-flow analysis of Android applications in DroidSafe. In: Proceedings 2015 Network and Distributed System Security Symposium. http://dx.doi.org/10.14722/ndss.2015.23089.

Gu, Y., Sun, K., Su, P., Li, Q., Lu, Y., Ying, L., Feng, D., 2017. JGRE: An analysis of JNI global reference exhaustion vulnerabilities in android. In: 2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, pp. 427–438. http://dx.doi.org/10.1109/DSN.2017.40.

Hao, S., Liu, B., Nath, S., Halfond, W.G., Govindan, R., 2014. PUMA: Programmable UI-automation for large-scale dynamic analysis of mobile apps. In: Proc. of the 12th Annual Int. Conf. on Mobile Systems, Applications, and Services. MobiSys '14, http://dx.doi.org/10.1145/2594368.2594390.

Hwang, S., Lee, S., Kim, J., Ryu, S., 2021. JUSTGen: Effective test generation for unspecified JNI behaviors on JVMs. In: 2021 43rd Int. Conf. on Soft. Eng.. ICSE '21, http://dx.doi.org/10.1109/ICSE43902.2021.00151.

Klieber, W., Flynn, L., Bhosale, A., Jia, L., Bauer, L., 2014. Android taint flow analysis for app sets. In: Proc. of the 3rd ACM SIGPLAN Int. Workshop on the State of the Art in Java Program Analysis. SOAP '14, http://dx.doi.org/10.1145/2614628.2614633.

Lee, S., Lee, H., Ryu, S., 2020. Broadening horizons of multilingual static analysis: Semantic summary extraction from C code for JNI program analysis. In: 2020 35th IEEE/ACM Int. Conf. on Automated Software Engineering. ASE'20, http://dx.doi.org/10.1145/3324884.3416558.

Li, L., Bartel, A., Bissyandé, T.F., Klein, J., Le Traon, Y., Arzt, S., Rasthofer, S., Bodden, E., Octeau, D., McDaniel, P., 2015. IccTA: Detecting inter-component privacy leaks in Android apps. In: 37th IEEE Int. Conf. on Soft. Eng. ASE'15, http://dx.doi.org/10.1109/ICSE.2015.48.

Li, S., Tan, G., 2009. Finding bugs in exceptional situations of JNI programs. In: Proceedings of the 16th ACM Conference on Computer and Communications Security. CCS '09, pp. 442–452. http://dx.doi.org/10.1145/1653662.1653716.

Liao, Y., Cai, R., Zhu, G., Yin, Y., Li, K., 2018. MobileFindr: Function similarity identification for reversing mobile binaries. In: ESORICS 2018: Computer Security. http://dx.doi.org/10.1007/978-3-319-99073-6_4.

Ming, J., Xu, D., Jiang, Y., Wu, D., 2017. BinSim: Trace-based semantic binary diffing via system call sliced segment equivalence checking. In: 26th USENIX Security Symposium. USENIX Security 17.

NSA, 2016. Ghidra. https://ghidra-sre.org/.

Oxgalz, 2021. Virtuailor. URL: https://github.com/0xgalz/Virtuailor.

Pawlowski, A., Contag, M., van der Veen, V., Ouwehand, C., Holz, T., Bos, H., Athanasopoulos, E., Giuffrida, C., 2017. MARX: uncovering class hierarchies in C++ programs. In: 24th Annual Network and Distributed System Security Symposium. NDSS '17.

Schwartz, E.J., Cohen, C.F., Duggan, M., Gennari, J., Havrilla, J.S., Hines, C., 2018. Using logic programming to recover C++ classes and methods from compiled executables. In: Proc. of the 2018 ACM SIGSAC Conf. on Comp. and Com. Sec.. CCS '18, http://dx.doi.org/10.1145/3243734.3243793.

Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen, J., Feng, S., Hauser, C., Krügel, C., Vigna, G., 2016. SOK: (state of) the art of war: Offensive techniques in binary analysis. In: IEEE SP'16. http://dx.doi.org/10.1109/SP.2016.17.

sirmabus, 2021. IDA CI plugin. URL: https://sourceforge.net/projects/classinformer/.

Strelsky, A., 2021. Ghidra C++ class and run time type information analyzer. URL: https://github.com/astrelsky/Ghidra-Cpp-Class-Analyzer.

Sun, M., Wei, T., Lui, J.C., 2016. TaintART: A practical multi-level information-flow tracking system for Android RunTime. In: Proc. of the 2016 Conf. on Comp. and Com. Sec.. CCS '16, http://dx.doi.org/10.1145/2976749.2978343.

Tan, G., Chakradhar, S., Srivaths, R., Wang, R.D., 2006. Safe Java native interface. In: In Proceedings of the 2006 IEEE International Symposium on Secure Software Engineering. pp. 97–106.

Tan, G., Croft, J., 2008. An Empirical Security Study of the Native Code in the JDK. In: Proc. of the 17th Conf. on Security Symposium. SS '08, USENIX, http://dx.doi.org/10.5555/1496711.1496736.

Wang, X., Zhu, S., Zhou, D., Yang, Y., 2017. Droid-AntiRM: Taming control flow anti-analysis to support automated dynamic analysis of Android malware. In: Proc. of the 33rd Annual Computer Security Applications Conference. http://dx.doi.org/10.1145/3134600.3134601.

Wei, F., Lin, X., Ou, X., Chen, T., Zhang, X., 2018a. JN-SAF: Precise and efficient NDK/JNI-Aware inter-language static analysis framework for security vetting of Android applications with native code. In: Proc. of the 2018 ACM SIGSAC Conf. on Computer and Communications Security. CCS '18, http://dx.doi.org/10.1145/3243734.3243835.

Wei, F., Roy, S., Ou, X., Robby, 2018b. Amandroid: A precise and general inter-component data flow analysis framework for security vetting of Android apps. ACM Trans. Priv. Secur. http://dx.doi.org/10.1145/3183575.

Wen, E., Zhou, J., Luo, X., Russello, G., Dietrich, J., 2024. Keep me updated: An empirical study on embedded JavaScript engines in android apps. In: 2024 IEEE/ACM 21st International Conference on Mining Software Repositories. MSR, pp. 361–372.

Wong, M.Y., Lie, D., 2016. IntelliDroid: A targeted input generator for the dynamic analysis of Android malware. In: Proceedings 2016 Network and Distributed System Security Symposium. http://dx.doi.org/10.14722/ndss.2016.23118.

Xu, Y., Xu, Z., Chen, B., Song, F., Liu, Y., Liu, T., 2020. Patch based vulnerability matching for binary programs. In: Proc. of the 29th ACM SIGSOFT Int. Symp. on Software Testing and Analysis. ISSTA '20, http://dx.doi.org/10.1145/3395363.3397361.

Xue, L., Qian, C., Zhou, H., Luo, X., Zhou, Y., Shao, Y., Chan, A.T., 2019. NDroid: Toward tracking information flows across multiple Android contexts. IEEE Trans. Inf. Forensics Secur. http://dx.doi.org/10.1109/TIFS.2018.2866347.

Xue, L., Zhou, Y., Chen, T., Luo, X., Gu, G., 2017. Malton: Towards on-device non-invasive mobile malware analysis for ART. In: 26th USENIX Security Symposium. USENIX Security 17, USENIX Association.

Yan, L.K., Yin, H., 2012. DroidScope: Seamlessly reconstructing the OS and Dalvik semantic views for dynamic Android malware analysis. In: 21st USENIX Security Symposium. USENIX Security 12.

Yuanchun Li, Ziyue Yang, Yao Guo, Xiangqun Chen, 2017. DroidBot: a lightweight UI-Guided test input generator for Android. In: 2017 IEEE/ACM 39th Int. Conf. on Software Engineering. http://dx.doi.org/10.1109/ICSE-C.2017.8.

Zhang, X., Heaps, J., Slavin, R., Niu, J., Breaux, T., Wang, X., 2023. DAISY: Dynamic-analysis-induced source discovery for sensitive data. ACM Trans. Softw. Eng. Methodol. 32 (4), http://dx.doi.org/10.1145/3569936.